

Tarefa 3 · Guia completo

A Tarefa 3 inteira, em cinco passos por bloco

Os 21 itens do roteiro, arquivo por arquivo. Em cada bloco: a lógica, o pseudocódigo, o código da Tarefa 2, como está hoje no seu `tarefa-3/src` e como deve ficar.

Dupla	Adriann e Alex
Situação	etapas A e B quase prontas; C, D e E por fazer (ver tabela da seção 2)
Verificação	o código final (passo 5) compila com o comando oficial e passa em 42 testes
Entrega	hoje, 25/09/2026, 23h59

O sumário é clicável. Material preparado com auxílio de IA (Claude): usem-no para entender, confirmem a política da disciplina e declarem o uso na justificativa.

Sumário

1. Como ler	3
1.1 Mapa	4
2. Onde vocês estão	5
3. Etapa A · Fundações (itens 1 a 5)	6
Item 1 · StatusDemanda	6
Item 2 · StatusProduto	8
Item 3 · StatusMaquina	9
Item 4 · Auditavel	11
Item 5 · Cenario	12
4. Etapa B · Produtos (itens 6 a 8)	14
Item 6 · TipoMedicamento	14
Item 7 · Produto	16
Item 8 · Os três medicamentos	20
5. Etapa C · Pedido e máquinas (itens 9 a 12)	24
Item 9 · Demanda	24
Item 10 · Maquina	29
Item 11 · As três máquinas	36
Item 12 · MateriaPrima	41
6. Etapa D · O padrão Strategy (itens 13 a 16)	43
Item 13 · EstrategiaProducao	43
Itens 14 a 16 · As três estratégias	45
7. Etapa E · Juntar tudo (itens 17 a 21)	48
Item 17 · Terminal	48
Item 18 · GerenciadorProducao	51
Item 19 · ConfiguradorFabrica	68
Item 20 · MenuFabrica	70
Item 21 · Main	79
8. Compilar, testar e entregar	80
8.1 Durante as etapas	80
8.2 No fim	80
8.3 Entrega	80

1. Como ler

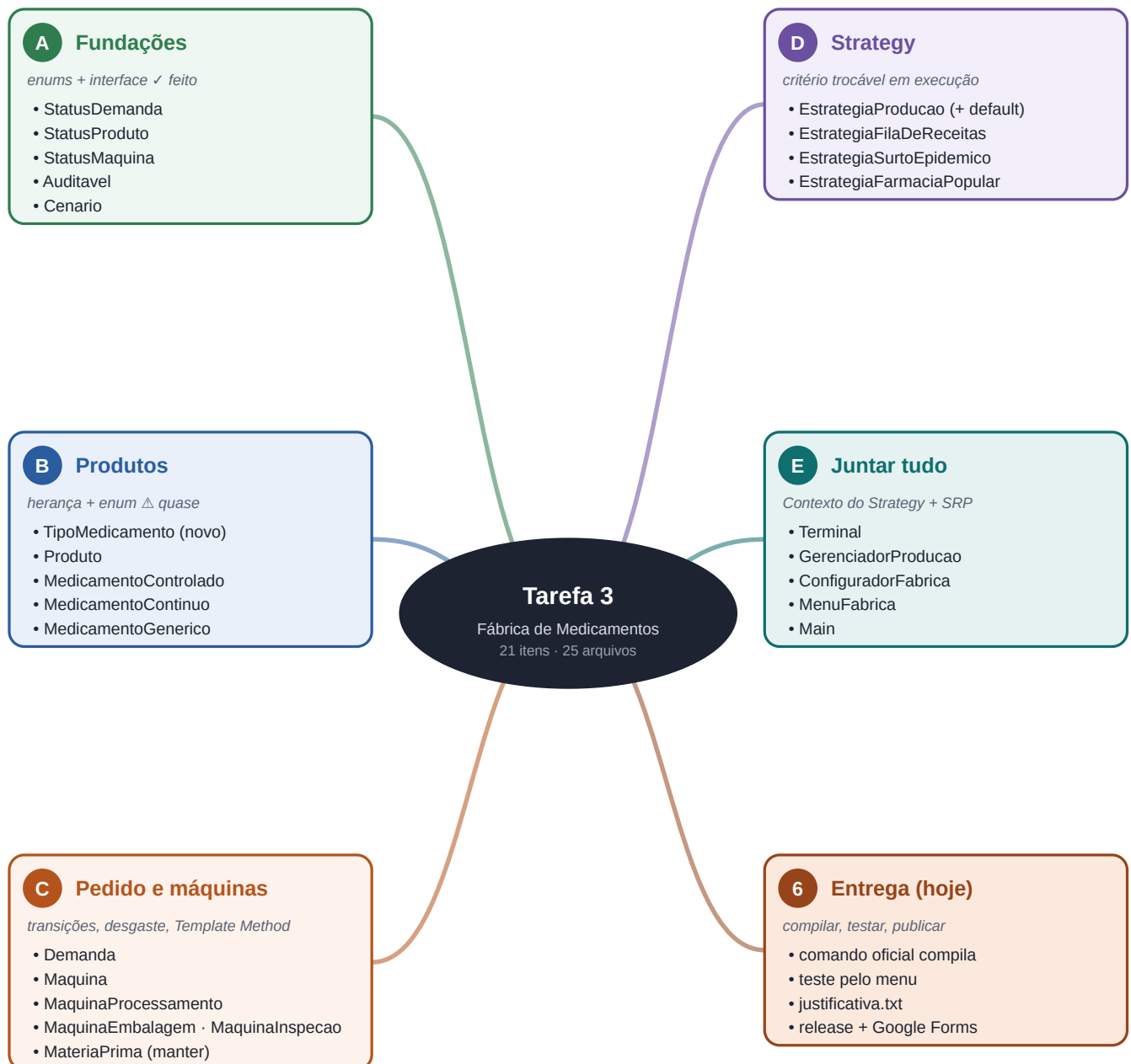
[↑ voltar ao sumário](#)

O guia segue a ordem de implementação (itens 1 a 21). Cada item é dividido em blocos, e cada bloco tem sempre os cinco passos:

Passo	O que mostra
1 · Lógica	o raciocínio em texto: o que muda e por quê.
2 · Pseudocódigo	a mesma ideia, sem a sintaxe do Java.
3 · Código antigo	como estava em <code> tarefa-2/src </code> (fundo vermelho claro).
4 · Como está agora	como está hoje em <code> tarefa-3/src </code> (fundo amarelo). Se ainda não mexeram, o passo diz isso.
5 · Como deve ficar	o código final (fundo cinza). Se já está certo, aparece ✓ Já está certo.

Marcas: MANTÉM MUDA SAI TAREFA 3 .

1.1 Mapa



2. Onde vocês estão

[↑ voltar ao sumário](#)

Comparando os arquivos de hoje com o código final:

Item	Arquivo	Situação
Item 1 · StatusDemanda	StatusDemanda.java	✓ pronto
Item 2 · StatusProduto	StatusProduto.java	✓ pronto
Item 3 · StatusMaquina	StatusMaquina.java	⚠ um ajuste
Item 4 · Auditavel	Auditavel.java	✓ pronto
Item 5 · Cenario	Cenario.java	✓ pronto
Item 6 · TipoMedicamento	TipoMedicamento.java	✓ pronto
Item 7 · Produto	Produto.java	⚠ um ajuste
Item 8 · Os três medicamentos	Medicamento*.java	⚠ 2 de 3 faltam
Item 9 · Demanda	Demanda.java	✗ ainda da Tarefa 2
Item 10 · Maquina	Maquina.java	✗ ainda da Tarefa 2
Item 11 · As três máquinas	Maquina*.java	✗ ainda da Tarefa 2
Item 12 · MateriaPrima	MateriaPrima.java	✓ não muda
Item 13 · EstrategiaProducao	EstrategiaProducao.java	⚠ falta um método
Itens 14 a 16 · As três estratégias	Estrategia*.java	✗ falta criar
Item 17 · Terminal	Terminal.java	✗ falta criar
Item 18 · GerenciadorProducao	GerenciadorProducao.java	✗ ainda da Tarefa 2
Item 19 · ConfiguradorFabrica	ConfiguradorFabrica.java	✗ falta criar
Item 20 · MenuFabrica	MenuFabrica.java	✗ falta criar
Item 21 · Main	Main.java	✗ ainda da Tarefa 2

Os dois ajustes rápidos que destravam as etapas A e B:

- Produto.java: precisaManuntecao → precisaManutencao (item 7).
- MedicamentoContinuo e MedicamentoGenerico: passar para o formato novo (item 8).

Depois disso, sigam a etapa C.

3. Etapa A · Fundações (itens 1 a 5)

[↑ voltar ao sumário](#)

Arquivos que não dependem de nenhum outro. Vocês já fizeram esta etapa.

Item 1 · StatusDemanda

✓ pronto StatusDemanda.java

O enum do status do pedido TAREFA 3

1 Lógica

Na Tarefa 2 o pedido só tinha `boolean atendida`: sim ou não. Mas um pedido passa por quatro situações (pendente, em produção, concluído, cancelado), e um `boolean` não cabe isso. O enum fecha a lista: o compilador não aceita outro valor. O `mudaPara` guarda as regras de quem pode ir para onde, perto dos próprios estados.

2 Pseudocódigo

```
enum StatusDemanda: PENDENTE, EM_PRODUCAO, CONCLUIDA, CANCELADA
    cada um guarda um rótulo (texto para a tela)
mudaPara(novo):
    PENDENTE    -> pode ir para EM_PRODUCAO ou CANCELADA
    EM_PRODUCAO -> só para CONCLUIDA
    CONCLUIDA   -> volta para PENDENTE (quando a quantidade é atualizada)
    CANCELADA   -> para nenhum
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 private boolean atendida;
```

Era só este atributo dentro da `Demanda`.

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1 public enum StatusDemanda {
2     PENDENTE("Pendente", "Aguardando a produção"),
3     EM_PRODUCAO("Em produção", "Lote na esteira de fabricação"),
4     CONCLUIDA("Concluída", "Todas as unidades foram fabricadas"),
5     CANCELADA("Cancelada", "Não tem matéria prima ou orçamento disponível");
6
7     private final String Rotulo;
8     private final String Detalhe;
9
10    StatusDemanda(String Rotulo, String Detalhe){
11        this.Rotulo = Rotulo;
12        this.Detalhe = Detalhe;
13    }
14
15    // Texto curto
16    public String getDescricao(){
17        return Rotulo;
18    }
19
20    // Texto longo
21    public String getDetalhe(){
22        return Detalhe;
23    }
24
25    // Agora, vou montar a lógica de transições para cada Status
26    public boolean mudaPara(StatusDemanda Novo){
27        switch (this) {
28            case PENDENTE:
```

```
29         return Novo == EM_PRODUCAO || Novo == CANCELADA;
30     case EM_PRODUCAO:
31         return Novo == CONCLUIDA;
32     case CONCLUIDA:
33         // Mesmo se produzir a quantidade de uma demanda pedida, posso precisar produzir a mesma
quantidade n vezes
34         // ou tive defeito na produção e assim devo produzir mais e por isso saio de concluida
para pendente.
35         return Novo == PENDENTE;
36     default:
37         return false;
38     }
39 }
40 }
```

5 Como deve ficar

✓ Já está certo: igual ao passo 4. Não precisa mexer.

Item 2 · StatusProduto

✓ pronto

StatusProduto.java

O enum do status do produto TAREFA 3

1 Lógica

Na Tarefa 2 o status era `String` e havia constantes como `APROVADO = "Aprovado"`. Um erro de digitação (`"Aprovdo"`) compilava e quebrava a lógica sem aviso. Com o enum, só existem os seis estados da linha de produção, na ordem em que o produto passa por eles.

2 Pseudocódigo

```
enum StatusProduto: AGUARDANDO, PROCESSANDO, PROCESSADO, EMBALADO, APROVADO, REJEITADO
    cada um guarda uma descrição (texto para a tela)
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 // Fazemos assim?
2 public static String AGUARDANDO = "Aguardando";
3 public static String APROVADO = "Aprovado";
4 public static String REJEITADO = "Rejeitado";
```

Ficava dentro do `Produto`.

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1 public enum StatusProduto {
2     AGUARDANDO("Aguardando"),
3     PROCESSADO("Processado"),
4     PROCESSANDO("Processando"),
5     EMBALADO("Embalado"),
6     APROVADO("Aprovado"),
7     REJEITADO("Rejeitado");
8
9     private final String Descricao;
10
11     StatusProduto(String Descricao){
12         this.Descricao = Descricao;
13     }
14
15     public String getDescricao(){
16         return Descricao;
17     }
18
19 }
```

5 Como deve ficar

✓ Já está certo: igual ao passo 4. Não precisa mexer.

Item 3 · StatusMaquina

⚠ um ajuste

StatusMaquina.java

O enum do estado da máquina TAREFA 3

1 Lógica

Na Tarefa 2 a máquina tinha `boolean ligada`. Na Tarefa 3 ela também pode estar **quebrada** (saúde chegou a zero), e "quebrada" não é o mesmo que "desligada": uma quebrada não pode ser ligada até o reparo. São três estados, então enum.

2 Pseudocódigo

```
enum StatusMaquina: DESLIGADA, LIGADA, QUEBRADA
    cada um guarda uma descrição
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 private boolean ligada;
```

Era só este atributo dentro da `Maquina`.

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1 public enum StatusMaquina {
2     DESLIGADA("Desligada"),
3     LIGADA("Ligada"),
4     QUEBRADA("Quebrada"),
5     MANUNTENCAO("Em manutenção");
6
7     private final String Descricao;
8
9     StatusMaquina(String Descricao){
10         this.Descricao = Descricao;
11     }
12
13     public String getDescricao(){
14         return Descricao;
15     }
16 }
```

5 Como deve ficar

tarefa-3 · final

```
1 public enum StatusMaquina {
2     DESLIGADA("Desligada"),
3     LIGADA("Ligada"),
4     QUEBRADA("Quebrada");
5
6     private final String Descricao;
7
8     StatusMaquina(String Descricao){
9         this.Descricao = Descricao;
10    }
11
12    public String getDescricao(){
13        return Descricao;
14    }
15 }
```

O ajuste: apaguem a constante `MANUNTENCAO("Em manuntenção")`. Ela está escrita errado e nenhum código usa: a máquina que precisa de manutenção é descoberta pelo método `precisaManutencao()`, não por um estado. Lembrem de trocar a vírgula depois de `QUEBRADA("Quebrada")` por ponto e vírgula.

Item 4 · Auditavel

✓ pronto

Auditavel.java

A interface **TAREFA 3**

1 Lógica

Máquinas e produtos são hierarquias diferentes (um não é filho do outro), mas o enunciado quer tratá-los do mesmo jeito numa auditoria. Uma interface resolve: é um contrato de "sei gerar meu relatório" e "sei dizer se preciso de atenção". Quem assina (`implements`) é obrigado a ter os dois métodos, com esses nomes exatos.

2 Pseudocódigo

```
interface Auditavel:
    gerarRelatorioDiagnostics() -> texto
    precisaManutencao()         -> verdadeiro/falso
```

3 Código antigo (Tarefa 2)

Não existia na Tarefa 2.

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1 public interface Auditavel {
2     String gerarRelatorioDiagnostics();
3     boolean precisaManutencao();
4 }
```

5 Como deve ficar

✓ Já está certo: igual ao passo 4. Não precisa mexer.

Atenção: o nome é `precisaManutencao`. Nas classes que implementam, qualquer letra diferente vira outro método e o Java acusa "must implement the inherited abstract method" (foi o erro do `Produto`).

Item 5 · Cenário

✓ pronto

Cenario.java

O enum dos cenários TAREFA 3

1 Lógica

Na Tarefa 2 os números da fábrica estavam soltos no `Main` (R\$ 1000, 500 kg). O enunciado pede dois cenários (normal e crise) que mudam esses números e também o desgaste e as falhas das máquinas. Cada cenário vira uma constante do enum com todos os seus valores juntos. A `sementeAleatoria` faz o sorteio sair igual toda vez no mesmo cenário, o que facilita testar.

2 Pseudocódigo

```
enum Cenario:
    FABRICA_NORMAL (budget 1000, insumo 100 kg, falha x0.5, desgaste 0 a 3, semente 42)
    FABRICA_PANDEMIA (budget 500, insumo 10 kg, falha x2.0, desgaste 5 a 15, semente 2026)
    cada constante guarda: nome, descrição, budget, insumo, fator de falha,
                          desgaste mínimo e máximo, semente
    um getter para cada valor
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1      MateriaPrima materiaPrima = new MateriaPrima(1, "Princípio Ativo Farmacêutico (Insumo)", 500.0,
"kg", 10.0);
2      GerenciadorProducao gerenciador = new GerenciadorProducao(materiaPrima, 1000.0);
```

Os valores estavam escritos direto no `Main`.

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1  public enum Cenario {
2
3      FABRICA_NORMAL(
4          "Fabrica em estado normal",
5          "Produto novo no mercado, já auditado pela Anvisa e com perspectiva de altas vendas",
6          1000.0,
7          100.0,
8          // metade da chance de falhar, por exemplo
9          0.5,
10         // saude/uso
11         0.0,
12         3.0,
13         42L),
14
15     FABRICA_PANDEMIA(
16         "Fabrica em estado Apocalíptico",
17         "Demanda nas alturas, falta de matéria prima, maquinas no limite e estoque de remédios no
vermelho",
18         // Diminui devido a primeira onda da pandemia
19         500.0,
20         // Diminui devido a alta procura
21         10.0,
22         // É o dobro ou mais, pois tenho que produzir rápido em pouco tempo com alta qualidade
23         2.0,
24         5.0,
25         15.0,
26         2026L);
27
28     private final String nome;
29     private final String descricao;
30     private final double budgetInicial;
```

```

31     private final double estoqueInicialMateriaPrima;
32     private final double fatorFalha;
33     private final double desgasteMinimo;
34     private final double desgasteMaximo;
35     private final long sementeAleatoria;
36
37     Cenario(String nome, String descricao, double budgetInicial,
38             double estoqueInicialMateriaPrima, double fatorFalha,
39             double desgasteMinimo, double desgasteMaximo, long sementeAleatoria){
40         this.nome = nome;
41         this.descricao = descricao;
42         this.budgetInicial = budgetInicial;
43         this.estoqueInicialMateriaPrima = estoqueInicialMateriaPrima;
44         this.fatorFalha = fatorFalha;
45         this.desgasteMinimo = desgasteMinimo;
46         this.desgasteMaximo = desgasteMaximo;
47         this.sementeAleatoria = sementeAleatoria;
48     }
49
50     public String getNome(){
51         return nome;
52     }
53     public String getDescricao(){
54         return descricao;
55     }
56     public double getBudgetInicial(){
57         return budgetInicial;
58     }
59     public double getEstoqueInicialMateriaPrima(){
60         return estoqueInicialMateriaPrima;
61     }
62     public double getFatorFalha(){
63         return fatorFalha;
64     }
65     public double getDesgasteMinimo(){
66         return desgasteMinimo;
67     }
68     public double getDesgasteMaximo(){
69         return desgasteMaximo;
70     }
71     public long getSementeAleatoria(){
72         return sementeAleatoria;
73     }
74
75 }

```

5 Como deve ficar

✓ Já está certo: igual ao passo 4. Não precisa mexer.

Lembrete: com só 10 kg de insumo, a Pandemia não consegue produzir nenhum pedido inicial sem comprar matéria-prima antes. Se for de propósito ("falta de matéria-prima"), expliquem isso na `justificativa.txt`.

4. Etapa B · Produtos (itens 6 a 8)

[↑ voltar ao sumário](#)

O `TipoMedicamento` precisa existir antes do `Produto`, e o `Produto` antes das filhas.

Item 6 · TipoMedicamento

✓ pronto `TipoMedicamento.java`

O enum que cria o produto certo TAREFA 3

1 Lógica

Na Tarefa 2 o gerenciador decidia o tipo pelo **índice** do pedido: `if (indiceDemanda == 0)` era Controlado, e o consumo (3, 2, 1 kg) ficava escrito ali. Agora cada tipo guarda os próprios dados (nome, sigla, consumo, qualidade) e sabe **criar** o seu medicamento: cada constante implementa `criarProduto`. Quem tem um tipo na mão chama `tipo.criarProduto(nome, lote)` e recebe o objeto certo, sem `if`.

2 Pseudocódigo

```
enum TipoMedicamento:
    CONTROLADO ("Controlado", "CTR", 3.0 kg/un, qualidade 0.9) -> cria MedicamentoControlado
    CONTINUO ("Contínuo", "CTN", 2.0 kg/un, qualidade 0.7) -> cria MedicamentoContínuo
    GENERICO ("Genérico", "GEN", 1.0 kg/un, qualidade 0.5) -> cria MedicamentoGenerico
    criarProduto(nome, lote): abstrato; cada constante escreve o seu
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 // Definir consumo de matéria-prima por unidade e tipo de produto
2 double consumoPorUnidade;
3 if (indiceDemanda == 0) {
4     consumoPorUnidade = 3.0;
5 } else if (indiceDemanda == 1) {
6     consumoPorUnidade = 2.0;
7 } else {
8     consumoPorUnidade = 1.0;
9 }
```

Ficava dentro do `fabricarDemanda` do gerenciador (e de novo mais abaixo, para escolher a classe).

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1 /* Por que preciso do TipoMedicamento?
2  Precisamos do TipoMedicamento, pois, na tarefa 2, o Gerenciador de produção
3  decidia o que fazer baseado no ÍNDICE da demanda "(if indiceDemanda == 0)..".
4  Diante disso, mediante leitura da teoria de Strategy definimos que
5  faz mais sentido deixar cada pedido carregar somente o seu nome e lote
6  e o disso o enum cria o Produto certo.
7  */
8
9 public enum TipoMedicamento {
10     CONTROLADO("Controlado", "CTR", 3.0, 0.9){
11         @Override
12         public Produto criarProduto(String nome, String lote){
13             return new MedicamentoControlado(nome, lote);
14         }
15     },
16     CONTINUO("Contínuo", "CTN", 2.0, 0.7){
17         @Override
18         public Produto criarProduto(String nome, String lote){
19             return new MedicamentoContínuo(nome, lote);
20         }
21     },
```

```

22     GENERICO("Genérico", "GEN", 1.0, 0.5){
23         @Override
24         public Produto criarProduto(String nome, String lote){
25             return new MedicamentoGenerico(nome, lote);
26         }
27     };
28
29     private final String nome;
30     private final String sigla;
31     private final double consumoPorUnidade; // kg/und
32     private final double qualidadePedida; // 0.0 (nenhuma) até 1.0 (perfeita)
33
34     TipoMedicamento(String nome, String sigla, double consumoPorUnidade, double qualidadePedida){
35         this.nome = nome;
36         this.sigla = sigla;
37         this.consumoPorUnidade = consumoPorUnidade;
38         this.qualidadePedida = qualidadePedida;
39     }
40
41     // Cada tipo cria o seu próprio medicamento usando nome e lote
42     public abstract Produto criarProduto(String nome, String lote);
43
44     // Gets
45     public String getNome(){
46         return nome;
47     }
48     public String getSigla(){
49         return sigla;
50     }
51     public double getConsumoPorUnidade(){
52         return consumoPorUnidade;
53     }
54     public double getQualidadePedida(){
55         return qualidadePedida;
56     }
57
58
59 }

```

5 Como deve ficar

✓ Já está certo: igual ao passo 4. Não precisa mexer.

Item 7 · Produto

⚠ um ajuste Produto.java

Detalhado linha a linha nos dois PDFs anteriores. Aqui, o resumo nos mesmos cinco passos.

Atributos e construtor MUDA

1 Lógica

O status vira `StatusProduto`; `qualidade`, `id` e `nome` ficam `final`. `Consumo` e `qualidade` saem do construtor porque vêm do `tipo`. O `id` deixa de vir de fora: o contador `static` soma 1 e o produto usa esse número. As três constantes `String` saem.

2 Pseudocódigo

```
atributos: id, nome, status (enum), qualidade, risco, contador static
           + lote, tipo (Tarefa 3)
construtor(nome, lote, tipo):
    contador <- contador + 1
    id <- contador
    nome, status <- AGUARDANDO, qualidade <- limitar(qualidade do tipo), risco <- 0
    guarda lote e tipo
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 public abstract class Produto {
2     // Atributos
3     private int id;
4     private String nome;
5     private String status;
6     private double quantidadeMateriaPrimaPorUnidade;
7     private double qualidade; // 0.0 a 1.0
8     private double probabilidadeFalhaAcumulada;
9     private static int totalProdutosFabricados = 0;
10
11     // Fazemos assim?
12     public static String AGUARDANDO = "Aguardando";
13     public static String APROVADO = "Aprovado";
14     public static String REJEITADO = "Rejeitado";
15
16     // Construtor
17     public Produto(int id, String nome, String status, double quantidadeMateriaPrimaPorUnidade, double
qualidade) {
18         this.id = id;
19         this.nome = nome;
20         this.status = status;
21         this.quantidadeMateriaPrimaPorUnidade = quantidadeMateriaPrimaPorUnidade;
22         this.qualidade = qualidade;
23         this.probabilidadeFalhaAcumulada = 0;
24         totalProdutosFabricados++;
25         this.qualidade = limitar(qualidade);
26     }
27 }
```

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1 public abstract class Produto implements Auditavel {
2     // Atributos
3     private final int id;
4     private final String nome;
5     private StatusProduto status;
6     private final double qualidade; // 0.0 a 1.0
```

```

7     private double probabilidadeFalhaAcumulada;
8     private static int totalProdutosFabricados = 0;
9
10    // Tarefa 3
11    private final String lote;
12    private final TipoMedicamento tipo;
13
14    // Construtor
15    protected Produto(String nome, String lote, TipoMedicamento tipo) {
16        // O produto gera seu id
17        totalProdutosFabricados++;
18        this.id = totalProdutosFabricados;
19        this.nome = nome;
20        this.status = StatusProduto.AGUARDANDO;
21        this.qualidade = limitar(tipo.getQualidadePedida());
22        this.probabilidadeFalhaAcumulada = 0.0;
23        // Tarefa3
24        this.lote = lote;
25        this.tipo = tipo;
26    }

```

5 Como deve ficar

✓ Já está certo: igual ao passo 4. Não precisa mexer.

Métodos abstratos, getters e setters MUDA

1 Lógica

Saem `processar()` (as três filhas repetiam o mesmo código) e o `getTipo()` abstrato (o tipo agora é atributo). Entra `getRiscoDeFalha()`. Saem os `set` de atributos `final` e os que deixavam colocar valores inválidos. `getStatus/setStatus` passam a usar o enum e `foiAprovado` compara com `==`.

2 Pseudocódigo

```

abstratos: calcularTempoProducao(quantidade), getRiscoDeFalha()
getters: id, nome, status, qualidade, risco, total
setStatus(StatusProduto)
limitar(valor): corta abaixo de 0 e acima de 1
aumentarProbabilidadeFalha(incremento): se > 0, soma e limita
foiAprovado: status == APROVADO

```

3 Código antigo (Tarefa 2)

tarefa-2

```

1     // Métodos Abstratos
2     public abstract void processar();
3     public abstract double calcularTempoProducao(int quantidade);
4     public abstract String getTipo();
5
6     // Métodos Concretos
7     public int getId() {
8         return id;
9     }

```

(mais os getters e setters de todos os atributos, e o `foiAprovado` com `APROVADO.equals(status)`)

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```

1     // Métodos Abstratos
2     public abstract double calcularTempoProducao(int quantidade);
3     // Tarefa3: cada tipo de produto sabe qual é o seu RiscoDeFalha
4     protected abstract double getRiscoDeFalha();
5
6     // Métodos Concretos
7     public int getId() {

```

```

8         return id;
9     }
10
11     public String getNome() {
12         return nome;
13     }
14
15     public StatusProduto getStatus() {
16         return status;
17     }
18
19     public void setStatus(StatusProduto status) {
20         this.status = status;
21     }
22
23     public double getQualidade() {
24         return qualidade;
25     }
26
27     public double getProbabilidadeFalhaAcumulada() {
28         return probabilidadeFalhaAcumulada;
29     }
30
31     public static int getTotalProdutosFabricados() {
32         return totalProdutosFabricados;
33     }
34
35     // Fixo valor entre 0.0 e 1.0
36     private static double limitar(double valor){
37         return Math.max(0.0, Math.min(1.0, valor));
38     }
39     public void aumentarProbabilidadeFalha(double incremento) {
40         if (incremento <= 0){
41             return;
42         }
43         probabilidadeFalhaAcumulada = limitar(probabilidadeFalhaAcumulada + incremento);
44     }
45
46     public boolean foiAprovado(){
47         return status == StatusProduto.APROVADO;
48     }

```

5 Como deve ficar

✓ Já está certo: igual ao passo 4. Não precisa mexer.

Novos no fim: getLote, getTipo, iniciarProcessamento e Auditavel TAREFA 3

1 Lógica

Getters dos atributos novos; `iniciarProcessamento` no lugar do `processar()` repetido; e os dois métodos que o `implements Auditavel` exige. Para um produto, "precisa de manutenção" é "precisa ser reanalisado": foi rejeitado ou o risco passou do limite do tipo.

2 Pseudocódigo

```

getLote -> lote ; getTipo -> tipo (enum)
iniciarProcessamento: status <- PROCESSANDO
precisaManutencao: status = REJEITADO ou risco >= getRiscoDeFalha()
gerarRelatorioDiagnostico: "Produto id | nome | lote | status | risco"

```

3 Código antigo (Tarefa 2)

Não existia na Tarefa 2.

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```

1  public String getLote(){
2      return lote;
3  }
4  public TipoMedicamento getTipo(){
5      return tipo;
6  }
7  // Substituo o processar()
8  public void iniciarProcessamento(){
9      status = StatusProduto.PROCESSANDO;
10 }
11 // Métodos da interface Auditavel
12 @Override
13 public boolean precisaManuntecao(){
14     return status == StatusProduto.REJEITADO ||
15         probabilidadeFalhaAcumulada >= getRiscoDeFalha();
16 }
17 @Override
18 public String gerarRelatorioDiagnostico(){
19     return "Produto " + id + " | " +
20         nome + " | lote " + lote
21         + " | " + status.getDescricao()
22         + " | risco " + probabilidadeFalhaAcumulada;
23 }
24 }
25 }

```

5 Como deve ficar

tarefa-3 · final

```

1  public String getLote(){
2      return lote;
3  }
4  public TipoMedicamento getTipo(){
5      return tipo;
6  }
7  // Substituo o processar()
8  public void iniciarProcessamento(){
9      status = StatusProduto.PROCESSANDO;
10 }
11 // Métodos da interface Auditavel
12 @Override
13 public boolean precisaManutencao(){
14     return status == StatusProduto.REJEITADO ||
15         probabilidadeFalhaAcumulada >= getRiscoDeFalha();
16 }
17 @Override
18 public String gerarRelatorioDiagnostico(){
19     return "Produto " + id + " | " +
20         nome + " | lote " + lote
21         + " | " + status.getDescricao()
22         + " | risco " + probabilidadeFalhaAcumulada;
23 }
24 }
25 }

```

O ajuste: no seu arquivo está `precisaManuntecao` (com "n" a mais). Tem que ser `precisaManutencao`, exatamente como no `Auditavel`. Enquanto não trocar, os três medicamentos mostram "must implement the inherited abstract method `Auditavel.precisaManutencao()`".

Item 8 · Os três medicamentos

⚠ 2 de 3 faltam Medicamento*.java

As três filhas mudam do mesmo jeito: construtor com (nome, lote), saem processar() e getTipo(), entra getRiscoDeFalha(). O relatório próprio é opcional; só o Controlado tem.

MedicamentoControlado MUDA

1 Lógica

O super passa o tipo CONTROLADO, de onde saem consumo (3 kg) e qualidade (0.9). É o mais rigoroso, então tolera o menor risco: 0.10. Completa o relatório do pai com o aviso da receita.

2 Pseudocódigo

```
construtor(nome, lote): super(nome, lote, CONTROLADO)
calcularTempoProducao(q): 10 * q
getRiscoDeFalha: 0.10
gerarRelatorioDiagnostico: relatório do pai + " | Precisa de uma receita especial controlada"
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 public class MedicamentoControlado extends Produto {
2
3     // Construtor
4     public MedicamentoControlado(int id, String nome, String status, double
quantidadeMateriaPrimaPorUnidade) {
5         super(id, nome, status, quantidadeMateriaPrimaPorUnidade, 0.9);
6     }
7
8     @Override
9     public void processar() {
10         setStatus("Processando");
11     }
12
13     @Override
14     public double calcularTempoProducao(int quantidade) {
15         return 10.0 * quantidade; // tempo(min)*quantidade(unidades)
16     }
17
18     @Override
19     public String getTipo() {
20         return "Medicamento de Alta Qualidade";
21     }
22 }
```

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1 // Ex: Ritalina
2
3 public class MedicamentoControlado extends Produto {
4     // Construtor
5     public MedicamentoControlado(String nome, String lote) {
6         super(nome, lote, TipoMedicamento.CONTROLADO);
7     }
8
9     @Override
10    public double calcularTempoProducao(int quantidade) {
11        return 10.0 * quantidade; // tempo(min)*quantidade(unidades)
12    }
13
14    @Override
15    protected double getRiscoDeFalha(){
```

```

16         return 0.10;
17     }
18
19     // Relatorio com base no texto da super classe
20     @Override
21     public String gerarRelatorioDiagnostico(){
22         return super.gerarRelatorioDiagnostico() + " | Precisa de uma receita especial controlada";
23     }
24 }

```

5 Como deve ficar

✓ Já está certo: igual ao passo 4. Não precisa mexer.

MedicamentoContínuo MUDA

1 Lógica

Mesma mudança, com o tipo CONTÍNUO e risco tolerado de 0.15. Não precisa de relatório próprio: herda o do Produto.

2 Pseudocódigo

```

construtor(nome, lote): super(nome, lote, CONTÍNUO)
calcularTempoProducao(q): 6 * q
getRiscoDeFalha: 0.15

```

3 Código antigo (Tarefa 2)

tarefa-2

```

1 public class MedicamentoContínuo extends Produto {
2
3     // Construtor
4     public MedicamentoContínuo(int id, String nome, String status, double
quantidadeMateriaPrimaPorUnidade) {
5         super(id, nome, status, quantidadeMateriaPrimaPorUnidade, 0.7);
6     }
7
8     @Override
9     public void processar() {
10         setStatus("Processando");
11     }
12
13     @Override
14     public double calcularTempoProducao(int quantidade) {
15         return 6.0 * quantidade; // tempo(min)*quantidade(unidades)
16     }
17
18     @Override
19     public String getTipo() {
20         return "Medicamento de Média Qualidade";
21     }
22 }

```

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```

1 public class MedicamentoContínuo extends Produto {
2
3     // Construtor
4     public MedicamentoContínuo(int id, String nome, String status, double
quantidadeMateriaPrimaPorUnidade) {
5         super(id, nome, status, quantidadeMateriaPrimaPorUnidade, 0.7);
6     }
7
8     @Override
9     public void processar() {
10         setStatus("Processando");

```

```

11     }
12
13     @Override
14     public double calcularTempoProducao(int quantidade) {
15         return 6.0 * quantidade; // tempo(min)*quantidade(unidades)
16     }
17
18     @Override
19     public String getTipo() {
20         return "Medicamento Contínuo";
21     }
22 }

```

5 Como deve ficar

tarefa-3 · final

```

1 // Ex: Losartana
2 public class MedicamentoContinuo extends Produto {
3     // Construtor
4     public MedicamentoContinuo(String nome, String lote) {
5         super(nome, lote, TipoMedicamento.CONTINUO);
6     }
7
8     @Override
9     public double calcularTempoProducao(int quantidade) {
10         return 6.0 * quantidade; // tempo(min)*quantidade(unidades)
11     }
12
13     // Tarefa 3: risco a partir do qual a unidade precisa ser reanalisada
14     @Override
15     protected double getRiscoDeFalha() {
16         return 0.15;
17     }
18 }

```

MedicamentoGenerico MUDA

1 Lógica

Mesma mudança, com o tipo `GENERICO` e risco tolerado de 0.20 (o menos rigoroso).

2 Pseudocódigo

```

construtor(nome, lote): super(nome, lote, GENERICO)
calcularTempoProducao(q): 5 * q
getRiscoDeFalha: 0.20

```

3 Código antigo (Tarefa 2)

tarefa-2

```

1 public class MedicamentoGenerico extends Produto {
2
3     // Construtor
4     public MedicamentoGenerico(int id, String nome, String status, double
quantidadeMateriaPrimaPorUnidade) {
5         super(id, nome, status, quantidadeMateriaPrimaPorUnidade, 0.5);
6     }
7
8     @Override
9     public void processar() {
10         setStatus("Processando");
11     }
12
13     @Override
14     public double calcularTempoProducao(int quantidade) {
15         return 5.0 * quantidade; // tempo(min)*quantidade(unidades)
16     }
17 }

```

```

18     @Override
19     public String getTipo() {
20         return "Medicamento de Baixa Qualidade";
21     }
22 }

```

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```

1  public class MedicamentoGenerico extends Produto {
2
3      // Construtor
4      public MedicamentoGenerico(int id, String nome, String status, double
quantidadeMateriaPrimaPorUnidade) {
5          super(id, nome, status, quantidadeMateriaPrimaPorUnidade, 0.5);
6      }
7
8      @Override
9      public void processar() {
10         setStatus("Processando");
11     }
12
13     @Override
14     public double calcularTempoProducao(int quantidade) {
15         return 5.0 * quantidade; // tempo(min)*quantidade(unidades)
16     }
17
18     @Override
19     public String getTipo() {
20         return "Medicamento Genérico";
21     }
22 }

```

5 Como deve ficar

tarefa-3 · final

```

1  // Ex: Dipirona
2  public class MedicamentoGenerico extends Produto {
3      // Construtor
4      public MedicamentoGenerico(String nome, String lote) {
5          super(nome, lote, TipoMedicamento.GENERICO);
6      }
7
8      @Override
9      public double calcularTempoProducao(int quantidade) {
10         return 5.0 * quantidade; // tempo(min)*quantidade(unidades)
11     }
12
13     // Tarefa 3: risco a partir do qual a unidade precisa ser reanalisada
14     @Override
15     protected double getRiscoDeFalha() {
16         return 0.20;
17     }
18 }

```

5. Etapa C · Pedido e máquinas (itens 9 a 12)

[↑ voltar ao sumário](#)

A `Demanda` vem antes das estratégias porque elas usam os métodos novos dela.

Item 9 · Demanda

X ainda da Tarefa 2 `Demanda.java`

O pedido deixa de ser identificado pela posição na lista e passa a ter número, tipo, nome comercial e status. As estratégias vão escolher pedidos, então o pedido precisa saber responder: "posso ser escolhido?", "quanto insumo gasto?", "quanto custo?".

Atributos e construtor **MUDA**

1 Lógica

`tipoProduto` (texto) vira `TipoMedicamento`; `atendida` vira `StatusDemanda`. Entram `numero` (ordem de chegada, gerado como o id do produto), `medicamento` (nome comercial) e o custo por unidade da linha. O construtor recusa tipo nulo e quantidade negativa.

2 Pseudocódigo

```
contador static proximoNumero = 1
atributos: numero, tipo, medicamento, custoPorUnidade, quantidade, status
construtor(tipo, medicamento, quantidade, custoPorUnidade):
    se tipo nulo ou quantidade < 0: erro
    numero <- proximoNumero ; proximoNumero + 1
    guarda os valores ; status <- PENDENTE
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 public class Demanda {
2
3     // Atributos
4     private String tipoProduto;
5     private int quantidadeProdutos;
6     private boolean atendida;
7
8     // Construtor
9     public Demanda(String tipoProduto, int quantidadeProdutos) {
10         this.tipoProduto = tipoProduto;
11         this.quantidadeProdutos = quantidadeProdutos;
12         this.atendida = false;
13     }
```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```
1 public class Demanda {
2
3     private static int proximoNumero = 1;
4
5     private final int numero;           // ordem de chegada (usada pela fila FIFO)
6     private final TipoMedicamento tipo;
7     private final String medicamento;   // nome comercial, ex.: "Paracetamol 500mg"
8     private final double custoOperacionalPorUnidade;
9     private int quantidade;
10    private StatusDemanda status;
11 }
```

```

12     public Demanda(TipoMedicamento tipo, String medicamento, int quantidade,
13                   double custoOperacionalPorUnidade) {
14         if (tipo == null) {
15             throw new IllegalArgumentException("O tipo do medicamento é obrigatório.");
16         }
17         if (quantidade < 0) {
18             throw new IllegalArgumentException("A quantidade não pode ser negativa.");
19         }
20         this.numero = proximoNumero++;
21         this.tipo = tipo;
22         this.medicamento = medicamento;
23         this.quantidade = quantidade;
24         this.custoOperacionalPorUnidade = custoOperacionalPorUnidade;
25         this.status = StatusDemanda.PENDENTE;
26     }

```

Mudanças de status MUDA

1 Lógica

O antigo `atender()` vira três métodos com nome de ação. Todos passam por **um** método privado, `mudarStatus`, que pergunta ao enum se a mudança é permitida (`mudaPara`) e, se não for, lança exceção. Assim ninguém conclui um pedido cancelado.

2 Pseudocódigo

```

iniciarProducao: mudarStatus(EM_PRODUCAO)
concluir:        mudarStatus(CONCLUIDA)
cancelar:        mudarStatus(CANCELADA)
mudarStatus(novo):
    se status.mudaPara(novo) é falso: erro "transição inválida"
    status <- novo

```

3 Código antigo (Tarefa 2)

tarefa-2

```

1     public void atender() {
2         this.atendida = true;
3     }

```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```

1     public void iniciarProducao() {
2         mudarStatus(StatusDemanda.EM_PRODUCAO);
3     }
4
5     public void concluir() {
6         mudarStatus(StatusDemanda.CONCLUIDA);
7     }
8
9     public void cancelar() {
10        mudarStatus(StatusDemanda.CANCELADA);
11    }
12
13    /** Ponto único de mudança de estado: garante a integridade das transições. */
14    private void mudarStatus(StatusDemanda novo) {
15        if (!status.mudaPara(novo)) {
16            throw new IllegalStateException(String.format(
17                "Transição inválida no pedido #d: %s -> %s",
18                numero, status.getDescricao(), novo.getDescricao()));
19        }

```

```
20     this.status = novo;
21 }
```

Atualizar a quantidade MUDA

1 Lógica

Só dá para mudar a quantidade de um pedido pendente ou já concluído (nunca de um em produção ou cancelado). Se estava concluído, a nova quantidade abre um novo ciclo: volta para PENDENTE.

2 Pseudocódigo

```
podeSerAtualizada: status = PENDENTE ou CONCLUIDA
atualizarQuantidade(nova):
    se nova < 0: erro
    se não podeSerAtualizada: erro
    quantidade <- nova
    se status = CONCLUIDA: mudarStatus(PENDENTE)
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 // Métodos
2 public void atualizarQuantidade(int novaQuantidade) {
3     this.quantidadeProdutos = novaQuantidade;
4     this.atendida = false;
5 }
```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```
1 /** Só pedidos PENDENTES ou CONCLUÍDOS podem receber nova quantidade. */
2 public boolean podeSerAtualizada() {
3     return status == StatusDemanda.PENDENTE || status == StatusDemanda.CONCLUIDA;
4 }
5
6 /** Nova quantidade = novo ciclo: um pedido concluído volta a ficar PENDENTE. */
7 public void atualizarQuantidade(int novaQuantidade) {
8     if (novaQuantidade < 0) {
9         throw new IllegalArgumentException("A quantidade não pode ser negativa.");
10    }
11    if (!podeSerAtualizada()) {
12        throw new IllegalStateException("Pedido #" + numero + " está "
13            + status.getDescricao() + " e não pode ser alterado.");
14    }
15    this.quantidade = novaQuantidade;
16    if (status == StatusDemanda.CONCLUIDA) {
17        mudarStatus(StatusDemanda.PENDENTE);
18    }
19 }
```

Consultas para as estratégias MUDA

1 Lógica

calcularMateriaPrimaNecessaria recebia o consumo por unidade de fora; agora o pedido sabe o próprio tipo e calcula sozinho. Entram o custo estimado e duas perguntas de sim/não que as estratégias usam.

2 Pseudocódigo

```
estaElegivel: status = PENDENTE e quantidade > 0
calcularConsumoEstimado: quantidade * consumo do tipo
calcularCustoEstimado: quantidade * custo por unidade
cabeNoOrcamento(caixa): custo estimado <= caixa
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 public double calcularMateriaPrimaNecessaria(double materiaPrimaPorUnidade) {
2     return materiaPrimaPorUnidade * quantidadeProdutos;
3 }
```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```
1  /** Pode ser escolhido por uma estratégia? */
2  public boolean estaElegivel() {
3      return status == StatusDemanda.PENDENTE && quantidade > 0;
4  }
5
6  /** kg de princípio ativo que o pedido vai consumir. */
7  public double calcularConsumoEstimado() {
8      return quantidade * tipo.getConsumoPorUnidade();
9  }
10
11 /** R$ que será debitado do orçamento para operar a linha. */
12 public double calcularCustoEstimado() {
13     return quantidade * custoOperacionalPorUnidade;
14 }
15
16 public boolean cabeNoOrcamento(double orcamentoDisponivel) {
17     return calcularCustoEstimado() <= orcamentoDisponivel;
18 }
```

Getters e toString MUDA

1 Lógica

Saem os `set`: número e tipo são `final`, e quantidade e status só mudam pelos métodos acima (que validam). O `toString` monta a linha que aparece no menu.

2 Pseudocódigo

```
getters de todos os atributos (sem setters)
toString: "Pedido #n - medicamento (tipo) xQtd [status]"
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1  // Getters e Setters
2  // TipoProduto
3  public String getTipoProduto() {
4      return tipoProduto;
5  }
6
7  public void setTipoProduto(String tipoProduto) {
8      this.tipoProduto = tipoProduto;
9  }
10
```

```

11 // QuantidadeProdutos
12 public int getQuantidadeProdutos() {
13     return quantidadeProdutos;
14 }
15
16 public void setQuantidadeProdutos(int quantidadeProdutos) {
17     this.quantidadeProdutos = quantidadeProdutos;
18 }
19
20 // Atendida
21 public boolean isAtendida() {
22     return atendida;
23 }
24
25 public void setAtendida(boolean atendida) {
26     this.atendida = atendida;
27 }
28 }

```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```

1 public int getNumero() { return numero; }
2 public TipoMedicamento getTipo() { return tipo; }
3 public String getMedicamento() { return medicamento; }
4 public int getQuantidade() { return quantidade; }
5 public StatusDemanda getStatus() { return status; }
6 public double getCustoOperacionalPorUnidade() { return custoOperacionalPorUnidade; }
7
8 @Override
9 public String toString() {
10     return String.format("Pedido #%d - %s (%s) x%d [%s]",
11         numero, medicamento, tipo.getNome(), quantidade, status.getDescricao());
12 }
13 }

```

Item 10 · Máquina

X ainda da Tarefa 2 Máquina.java

A máquina ganha saúde (0 a 100), desgaste por lote, chance de falha que cresce quando a saúde cai, estado como enum e a interface `Auditavel`.

Atributos e construtor MUDA

1 Lógica

boolean `ligada` vira `StatusMáquina estado`. Entram saúde, desgaste mínimo e máximo (vindos do cenário), limiar de manutenção, contadores de falhas e ciclos. A chance de falha base é multiplicada pelo fator do cenário. O `Random` vem de fora (com a semente do cenário) em vez de `new Random()`, para os resultados se repetirem.

2 Pseudocódigo

```
constantes: SAUDE_MAXIMA = 100 ; LIMIAR_PADRAO = 30
construtor(nome, capacidade, falhaBase, custo, cenário, random):
    falhaBase <- min(1, falhaBase * fator do cenário)
    desgaste mínimo e máximo <- do cenário
    estado <- DESLIGADA ; saude <- 100 ; limiar <- 30
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 public abstract class Máquina {
2     // Atributos
3     private String nome;
4     private boolean ligada;
5     private int capacidadeMaxima;
6     private double probabilidadeFalha;
7     private double custoOperacao;
8     protected Random random;
9
10    // Construtor (inicializo)
11    public Máquina(String nome, int capacidadeMaxima, double probabilidadeFalha, double custoOperacao) {
12        this.nome = nome;
13        this.ligada = false;
14        this.capacidadeMaxima = capacidadeMaxima;
15        this.probababilidadeFalha = probabilidadeFalha;
16        this.custoOperacao = custoOperacao;
17        this.random = new Random();
18    }
```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```
1 public abstract class Máquina implements Auditavel {
2
3     public static final double SAUDE_MAXIMA = 100.0;
4     public static final double LIMIAR_MANUTENCAO_PADRAO = 30.0;
5
6     private final String nome;
7     private final int capacidadeMaxima;
8     private final double probabilidadeFalhaBase; // já ajustada pelo cenário
9     private final double custoOperacao;        // R$ por unidade processada
10    private final double desgasteMinimo;
11    private final double desgasteMaximo;
12    private double limiarManutencao;             // configurável (padrão 30)
13    private StatusMáquina estado;
```

```

14     private double saude;
15     private int historicoFalhas;
16     private int ciclosDeUso;
17
18     /** Random compartilhado (com seed do cenário) para resultados reproduzíveis. */
19     protected final Random random;
20
21     protected Maquina(String nome, int capacidadeMaxima, double probabilidadeFalhaBase,
22                        double custoOperacao, Cenario cenario, Random random) {
23         this.nome = nome;
24         this.capacidadeMaxima = capacidadeMaxima;
25         this.probabilidadeFalhaBase = Math.min(1.0, probabilidadeFalhaBase * cenario.getFatorFalha());
26         this.custoOperacao = custoOperacao;
27         this.desgasteMinimo = cenario.getDesgasteMinimo();
28         this.desgasteMaximo = cenario.getDesgasteMaximo();
29         this.limiarManutencao = LIMIAR_MANUTENCAO_PADRAO;
30         this.random = random;
31         this.estado = StatusMaquina.DESLIGADA;
32         this.saude = SAUDE_MAXIMA;
33     }

```

processar: o passo comum fica na mãe MUDA

1 Lógica

Na Tarefa 2 cada filha repetia `if (produto == null || !estaLigada()) return;`. Agora a mãe faz essa checagem em `processar` (marcado `final`, ninguém sobrescreve) e chama `executarEtapa`, que cada filha implementa. Isso é o padrão *Template Method*.

2 Pseudocódigo

```

processar(produto):    (final)
    se produto nulo ou estado != LIGADA: sai
    executarEtapa(produto)
executarEtapa(produto): abstrato
getTipo(): abstrato

```

3 Código antigo (Tarefa 2)

tarefa-2

```

1     // Métodos Abstratos
2     public abstract void processar(Produto produto);
3     public abstract String getTipo();

```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```

1     /**
2      * Passo comum a todas as máquinas: só processa se estiver operando.
3      * "final" impede que uma subclasse esqueça essa checagem; cada subclasse
4      * implementa apenas a SUA etapa em executarEtapa().
5      */
6     public final void processar(Produto produto) {
7         if (produto == null || estado != StatusMaquina.LIGADA) {
8             return;
9         }
10        executarEtapa(produto);
11    }
12
13    protected abstract void executarEtapa(Produto produto);
14
15    public abstract String getTipo();

```

Ligar e desligar MUDA

1 Lógica

Uma máquina quebrada não pode ser ligada, então `ligar` devolve `false` nesse caso. `estaLigada` sai: quem precisa usa `getEstado()`.

2 Pseudocódigo

```
ligar: se QUEBRADA: devolve falso ; senão estado <- LIGADA, devolve verdadeiro
desligar: se LIGADA: estado <- DESLIGADA
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 // Métodos Concretos
2 public void ligar() {
3     this.ligada = true;
4 }
5
6 public void desligar() {
7     this.ligada = false;
8 }
9
10 public boolean estaLigada() {
11     return ligada;
12 }
```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```
1 /** @return false se a máquina está quebrada e não pode ser ligada. */
2 public boolean ligar() {
3     if (estado == StatusMaquina.QUEBRADA) {
4         return false;
5     }
6     estado = StatusMaquina.LIGADA;
7     return true;
8 }
9
10 public void desligar() {
11     if (estado == StatusMaquina.LIGADA) {
12         estado = StatusMaquina.DESLIGADA;
13     }
14 }
```

Desgaste e falhas MUDA

1 Lógica

A cada lote, a saúde cai um valor sorteado entre o desgaste mínimo e o máximo do cenário; em zero, a máquina quebra. A chance de falha cresce quando a saúde cai: com metade da saúde, o dobro da chance.

`verificarFalha` usa essa chance atual e conta as falhas.

2 Pseudocódigo

```
registrarCicloDeUso:
  desgaste <- sorteio entre mínimo e máximo
  saude <- max(0, saude - desgaste) ; ciclos + 1
  se saude = 0: estado <- QUEBRADA
getProbabilidadeFalhaAtual: se saude = 0: 1 ; senão min(1, base * 100 / saude)
verificarFalha: sorteio < chance atual ? (conta a falha)
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 // Método protegido com aleatoriedade
2 protected boolean verificarFalha() {
3     return random.nextDouble() < probabilidadeFalha;
4 }
```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```
1 /**
2  * Chamado uma vez por ciclo de fabricação (um lote).
3  * Reduz a saúde num valor aleatório entre desgasteMinimo e desgasteMaximo.
4  * @return quantos pontos de saúde foram perdidos.
5  */
6 public double registrarCicloDeUso() {
7     double desgaste = desgasteMinimo + random.nextDouble() * (desgasteMaximo - desgasteMinimo);
8     saude = Math.max(0.0, saude - desgaste);
9     ciclosDeUso++;
10    if (saude <= 0.0) {
11        estado = StatusMaquina.QUEBRADA;
12    }
13    return desgaste;
14 }
15
16 /**
17  * Chance de falha inversamente proporcional à saúde:
18  * saúde 100 -> chance base; saúde 50 -> 2x; saúde 25 -> 4x (limitada a 100%).
19  */
20 public double getProbabilidadeFalhaAtual() {
21     if (saude <= 0.0) {
22         return 1.0;
23     }
24     return Math.min(1.0, probabilidadeFalhaBase * (SAUDE_MAXIMA / saude));
25 }
26
27 /** Sorteia se a máquina falhou nesta unidade e registra no histórico. */
28 protected boolean verificarFalha() {
29     boolean falhou = random.nextDouble() < getProbabilidadeFalhaAtual();
30     if (falhou) {
31         historicoFalhas++;
32     }
33     return falhou;
34 }
```

Reparo e limiar TAREFA 3

1 Lógica

reparar devolve a saúde a 100 e deixa a máquina desligada (pronta para ligar). O enunciado pede um limiar de manutenção configurável: o set só aceita valores entre 0 e 100.

2 Pseudocódigo

```
reparar: saude <- 100 ; estado <- DESLIGADA
setLimiarManutencao(l): se 0 <= l <= 100: limiar <- l
estaQuebrada: estado = QUEBRADA
```

3 Código antigo (Tarefa 2)

Não existia na Tarefa 2.

4 Como está agora (seu arquivo)

Ainda não existe no seu `tarefa-3/src`.

5 Como deve ficar

tarefa-3 · final

```
1  /** Manutenção completa: saúde volta a 100 e a máquina pode ser ligada de novo. */
2  public void reparar() {
3      saude = SAUDE_MAXIMA;
4      estado = StatusMaquina.DESLIGADA;
5  }
6
7  /** O enunciado pede um limiar configurável para precisaManutencao(). */
8  public void setLimiarManutencao(double limiar) {
9      if (limiar >= 0 && limiar <= SAUDE_MAXIMA) {
10         this.limiarManutencao = limiar;
11     }
12 }
13
14 public boolean estaQuebrada() {
15     return estado == StatusMaquina.QUEBRADA;
16 }
```

Auditavel TAREFA 3

1 Lógica

Para a máquina, precisa de manutenção quem está quebrada **ou** com saúde abaixo do limiar. O relatório mostra saúde com uma barrinha, estado, chance de falha, falhas e ciclos.

2 Pseudocódigo

```
precisaManutencao: quebrada ou saude < limiar
gerarRelatorioDiagnostico: "[Máquina] nome | saúde [####....] | estado | falha | falhas | ciclos"
barraDeSaude: 10 casas; '#' para cada 10% de saúde
```

3 Código antigo (Tarefa 2)

Não existia na Tarefa 2.

4 Como está agora (seu arquivo)

Ainda não existe no seu `tarefa-3/src`.

5 Como deve ficar

tarefa-3 · final

```
1  @Override
2  public boolean precisaManutencao() {
3      return estaQuebrada() || saude < limiarManutencao;
4  }
5
6  @Override
7  public String gerarRelatorioDiagnostico() {
8      return String.format("[Máquina] %-22s | saúde %5.1f%% %s | %-28s | falha atual %4.1f%% | falhas %d
| ciclos %d%s",
9          nome, saude, barraDeSaude(), estado.getDescricao(),
```

```

10         getProbabilidadeFalhaAtual() * 100, historicoFalhas, ciclosDeUso,
11         precisaManutencao() ? " | MANUTENÇÃO NECESSÁRIA" : "");
12     }
13
14     /** Ex.: [#####...] para 70% de saúde. */
15     private String barraDeSaude() {
16         int cheios = (int) Math.round(saude / 10.0);
17         StringBuilder barra = new StringBuilder("");
18         for (int i = 0; i < 10; i++) {
19             barra.append(i < cheios ? '#' : '.');
20         }
21         return barra.append(']').toString();
22     }

```

Getters MUDA

1 Lógica

Os atributos agora são `final` ou só mudam por métodos que validam, então os `set` saem.

2 Pseudocódigo

getters de todos os atributos (sem setters)

3 Código antigo (Tarefa 2)

tarefa-2

```

1     public String getNome() {
2         return nome;
3     }
4
5     public void setNome(String nome) {
6         this.nome = nome;
7     }
8
9     public int getCapacidadeMaxima() {
10        return capacidadeMaxima;
11    }
12
13    public void setCapacidadeMaxima(int capacidadeMaxima) {
14        this.capacidadeMaxima = capacidadeMaxima;
15    }
16
17    public double getProbabilidadeFalha() {
18        return probabilidadeFalha;
19    }
20
21    public void setProbabilidadeFalha(double probabilidadeFalha) {
22        this.probabilidadeFalha = probabilidadeFalha;
23    }
24
25    public double getCustoOperacao() {
26        return custoOperacao;
27    }
28
29    public void setCustoOperacao(double custoOperacao) {
30        this.custoOperacao = custoOperacao;
31    }

```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```

1     public String getNome() { return nome; }
2     public int getCapacidadeMaxima() { return capacidadeMaxima; }
3     public double getCustoOperacao() { return custoOperacao; }

```

```
4     public double getProbabilidadeFalhaBase() { return probabilidadeFalhaBase; }
5     public StatusMaquina getEstado() { return estado; }
6     public double getSaude() { return saude; }
7     public int getHistoricoFalhas() { return historicoFalhas; }
8     public int getCiclosDeUso() { return ciclosDeUso; }
9     public double getLimiarManutencao() { return limiarManutencao; }
10 }
```

Item 11 · As três máquinas

X ainda da Tarefa 2

Maquina*.java

As regras de cada etapa são as da Tarefa 2. Muda o construtor (recebe cenário e `Random`), o método passa a ser `executarEtapa` (a checagem de ligada está na mãe) e o status vira enum.

MaquinaProcessamento MUDA

1 Lógica

Mistura o produto. Se a máquina falhar, o risco do produto sobe 0.10. Na Tarefa 2 havia um atributo separado (`chanceAumentarFalha`); agora a chance é a própria chance de falha da máquina, que depende da saúde.

2 Pseudocódigo

```
executarEtapa(produto):
    status <- PROCESSANDO
    se verificarFalha(): risco do produto + 0.10
    status <- PROCESSADO
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 public class MaquinaProcessamento extends Maquina {
2     private double chanceAumentarFalha; // Chance de X% de aumentar a probabilidade de falha do produto
3
4     // Construtor
5     public MaquinaProcessamento(String nome, int capacidadeMaxima, double probabilidadeFalha, double
custoOperacao, double chanceAumentarFalha) {
6         super(nome, capacidadeMaxima, probabilidadeFalha, custoOperacao);
7         this.chanceAumentarFalha = chanceAumentarFalha;
8     }
9
10    @Override
11    public void processar(Produto produto) {
12        if (produto == null || !estaLigada()){
13            return;
14        }
15        // Não falha diretamente, mas tem chanceAumentarFalha (X%) de aumentar a probabilidade de falha
acumulada do produto
16        if (random.nextDouble() < chanceAumentarFalha) {
17            produto.aumentarProbabilidadeFalha(0.10);
18        }
19        produto.setStatus("Processado");
20    }
21
22    @Override
23    public String getTipo() {
24        return "Máquina de Processamento / Mistura Farmacêutica";
25    }
26
27    public double getChanceAumentarFalha() {
28        return chanceAumentarFalha;
29    }
30
31    public void setChanceAumentarFalha(double chanceAumentarFalha) {
32        this.chanceAumentarFalha = chanceAumentarFalha;
33    }
34 }
```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```
1 import java.util.Random;
2
3 /**
4  * Reator de mistura. Quando falha, a mistura sai irregular e o risco do
5  * produto aumenta (mesma regra da Tarefa 2, agora ligada à saúde da máquina).
6  */
7 public class MaquinaProcessamento extends Maquina {
8
9     private static final double AUMENTO_RISCO = 0.10;
10
11     public MaquinaProcessamento(String nome, int capacidadeMaxima, double probabilidadeFalhaBase,
12                                   double custoOperacao, Cenario cenario, Random random) {
13         super(nome, capacidadeMaxima, probabilidadeFalhaBase, custoOperacao, cenario, random);
14     }
15
16     @Override
17     protected void executarEtapa(Produto produto) {
18         produto.setStatus(StatusProduto.PROCESSANDO);
19         if (verificarFalha()) {
20             produto.aumentarProbabilidadeFalha(AUMENTO_RISCO);
21         }
22         produto.setStatus(StatusProduto.PROCESSADO);
23     }
24
25     @Override
26     public String getTipo() {
27         return "Processamento / Mistura";
28     }
29 }
```

MaquinaEmbalagem MUDA

1 Lógica

Embala. Se falhar, a selagem sai ruim e o risco do produto sobe 0.05.

2 Pseudocódigo

```
executarEtapa(produto):
    se verificarFalha(): risco do produto + 0.05
    status <- EMBALADO
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 public class MaquinaEmbalagem extends Maquina {
2     private double chanceAumentarFalha; // Chance de Y% de aumentar a probabilidade de falha do produto
3
4     // Construtor
5     public MaquinaEmbalagem(String nome, int capacidadeMaxima, double probabilidadeFalha, double
6     custoOperacao, double chanceAumentarFalha) {
7         super(nome, capacidadeMaxima, probabilidadeFalha, custoOperacao);
8         this.chanceAumentarFalha = chanceAumentarFalha;
9     }
10
11     @Override
12     public void processar(Produto produto) {
13         if (produto == null || !estaLigada()){
14             return;
15         }
16         // Não falha diretamente, mas tem chanceAumentarFalha (Y%) de aumentar a probabilidade de falha
17         acumulada do produto
18         if (random.nextDouble() < chanceAumentarFalha) {
19             produto.aumentarProbabilidadeFalha(0.05);
20         }
21     }
22 }
```

```

19     produto.setStatus("Embalado");
20 }
21
22 @Override
23 public String getTipo() {
24     return "Máquina de Embalagem / Blister";
25 }
26
27 public double getChanceAumentarFalha() {
28     return chanceAumentarFalha;
29 }
30
31 public void setChanceAumentarFalha(double chanceAumentarFalha) {
32     this.chanceAumentarFalha = chanceAumentarFalha;
33 }
34 }

```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```

1  import java.util.Random;
2
3  /**
4   * Blistadeira (embalagem). Quando falha, a selagem sai imperfeita e o risco
5   * do produto aumenta um pouco.
6   */
7  public class MaquinaEmbalagem extends Maquina {
8
9      private static final double AUMENTO_RISCO = 0.05;
10
11     public MaquinaEmbalagem(String nome, int capacidadeMaxima, double probabilidadeFalhaBase,
12                             double custoOperacao, Cenario cenario, Random random) {
13         super(nome, capacidadeMaxima, probabilidadeFalhaBase, custoOperacao, cenario, random);
14     }
15
16     @Override
17     protected void executarEtapa(Produto produto) {
18         if (verificarFalha()) {
19             produto.aumentarProbabilidadeFalha(AUMENTO_RISCO);
20         }
21         produto.setStatus(StatusProduto.EMBALADO);
22     }
23
24     @Override
25     public String getTipo() {
26         return "Embalagem / Blister";
27     }
28 }

```

MaquinaInspecao MUDA

1 Lógica

Mesma regra da Tarefa 2: quanto mais rigoroso o remédio e maior o risco, maior a chance de rejeitar. Se a própria máquina falhar, a decisão sai invertida. Só o final muda: `setStatus` com o enum.

2 Pseudocódigo

```

chanceRejeicao <- risco do produto * (qualidade / 0.5)
defeituoso <- sorteio < chanceRejeicao
se a máquina falhou: aprovado <- defeituoso ; senão aprovado <- não defeituoso
status <- APROVADO ou REJEITADO

```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 public class MaquinaInspecao extends Maquina {
2
3     // Construtor
4     public MaquinaInspecao(String nome, int capacidadeMaxima, double probabilidadeFalha, double
custoOperacao) {
5         super(nome, capacidadeMaxima, probabilidadeFalha, custoOperacao);
6     }
7
8     @Override
9     public void processar(Produto produto) {
10         if (produto == null || !estaLigada()){
11             return;
12         }
13
14         // 1. Chance de rejeição baseada na probabilidade de falha acumulada e no rigor de qualidade:
15         // Alta qualidade (0.9) -> critérios mais rigorosos -> maior chance de rejeição
16         // Baixa qualidade (0.5) -> critérios mais flexíveis -> menor chance de rejeição
17         double chanceRejeicao = produto.getProbabilidadeFalhaAcumulada() * (produto.getQualidade() / 0.5);
18
19         boolean defeituoso = random.nextDouble() < chanceRejeicao;
20
21         // 2. A máquina de inspeção pode falhar diretamente com probabilidade Z% (verificarFalha()),
22         // resultando em uma inspeção incorreta (inverte a decisão de aprovação/rejeição)
23         boolean maquinaFalhou = verificarFalha();
24         boolean aprovado = maquinaFalhou ? defeituoso : !defeituoso;
25
26         if (aprovado) {
27             produto.setStatus("Aprovado");
28         } else {
29             produto.setStatus("Rejeitado");
30         }
31     }
32
33     @Override
34     public String getTipo() {
35         return "Máquina de Inspeção / Controle de Qualidade";
36     }
37 }
```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```
1 import java.util.Random;
2
3 /**
4  * Espectrômetro de controle de qualidade (inspeção "Anvisa").
5  * Regra da Tarefa 2 mantida: quanto maior o rigor (qualidade) do medicamento,
6  * maior a chance de rejeitar um produto com risco acumulado. Se a própria
7  * máquina falhar, a decisão sai invertida (falso positivo / falso negativo).
8  */
9 public class MaquinaInspecao extends Maquina {
10
11     private static final double QUALIDADE_REFERENCIA = 0.5;
12
13     public MaquinaInspecao(String nome, int capacidadeMaxima, double probabilidadeFalhaBase,
14                             double custoOperacao, Cenario cenario, Random random) {
15         super(nome, capacidadeMaxima, probabilidadeFalhaBase, custoOperacao, cenario, random);
16     }
17
18     @Override
19     protected void executarEtapa(Produto produto) {
20         double chanceRejeicao = produto.getProbabilidadeFalhaAcumulada()
21             * (produto.getQualidade() / QUALIDADE_REFERENCIA);
22         boolean defeituoso = random.nextDouble() < chanceRejeicao;
```

```
23
24     boolean maquinaFalhou = verificarFalha();
25     boolean aprovado = maquinaFalhou ? defeituoso : !defeituoso;
26
27     produto.setStatus(aprovado ? StatusProduto.APROVADO : StatusProduto.REJEITADO);
28 }
29
30 @Override
31 public String getTipo() {
32     return "Inspeção / Controle de Qualidade";
33 }
34 }
```

Item 12 · MateriaPrima

✓ não muda

MateriaPrima.java

A classe inteira **MANTÉM**

1 Lógica

Já estava bem encapsulada na Tarefa 2 (consumir, adicionar, verificar disponibilidade). A Tarefa 3 não pede mudança.

2 Pseudocódigo

sem mudança

3 Código antigo (Tarefa 2)

tarefa-2

```
1 // Qnt tenho de mp?
2
3 public class MateriaPrima {
4     // Atributos
5     private int id;
6     private String nome;
7     private double quantidade;
8     private String unidade;
9     private double custoPorUnidade;
10
11     // Construtor
12     public MateriaPrima(int id, String nome, double quantidade, String unidade, double custoPorUnidade) {
13         this.id = id;
14         this.nome = nome;
15         this.quantidade = quantidade;
16         this.unidade = unidade;
17         this.custoPorUnidade = custoPorUnidade;
18     }
19
20     // Métodos
21     public boolean consumir(double qtd) {
22         if (this.quantidade >= qtd) {
23             this.quantidade -= qtd;
24             return true;
25         }
26         return false;
27     }
28
29     public void adicionarEstoque(double qtd) {
30         this.quantidade += qtd;
31     }
32
33     public boolean verificarDisponibilidade(double qtdNecessaria) {
34         return this.quantidade >= qtdNecessaria;
35     }
36
37     // Getters e Setters
38     public int getId() {
39         return id;
40     }
41
42     public void setId(int id) {
43         this.id = id;
44     }
45
46     public String getNome() {
47         return nome;
48     }
49
50     public void setNome(String nome) {
```

```
51     this.nome = nome;
52 }
53
54 public double getQuantidade() {
55     return quantidade;
56 }
57
58 public void setQuantidade(double quantidade) {
59     this.quantidade = quantidade;
60 }
61
62 public String getUnidade() {
63     return unidade;
64 }
65
66 public void setUnidade(String unidade) {
67     this.unidade = unidade;
68 }
69
70 public double getCustoPorUnidade() {
71     return custoPorUnidade;
72 }
73
74 public void setCustoPorUnidade(double custoPorUnidade) {
75     this.custoPorUnidade = custoPorUnidade;
76 }
77 }
```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

✓ Já está certo: igual ao passo 4. Não precisa mexer.

Item 13 · EstrategiaProducao

⚠ falta um método EstrategiaProducao.java

A interface do Strategy **TAREFA 3**

1 Lógica

Na Tarefa 2 o usuário escolhia qual pedido fabricar (opções 4, 5 e 6). Agora a fábrica pode decidir sozinha, e o **critério** é trocável. Cada critério é uma classe que implementa esta interface. As três só consideram pedidos pendentes com quantidade maior que zero, então esse filtro fica num método `default`, escrito uma vez só.

2 Pseudocódigo

```
interface EstrategiaProducao:
    selecionarDemanda(pedidos, caixa) -> pedido ou null
    getNomeEstrategia() -> texto
    filtrarElegiveis(pedidos): (default)
        lista nova ; para cada pedido: se estaElegivel(): adiciona ; devolve a lista
```

3 Código antigo (Tarefa 2)

Não existia na Tarefa 2.

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public interface EstrategiaProducao {
5
6     // Retorno o pedido que será produzido ou null se não for possível produzir
7     Demanda selecionarDemanda(List<Demanda> demandas, double orcamentoDisponivel);
8
9     // Nome amigável que exibo para o usuário
10    String getNomeEstrategia();
11
12
13 }
```

5 Como deve ficar

tarefa-3 · final

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public interface EstrategiaProducao {
5
6     // Retorno o pedido que será produzido ou null se não for possível produzir
7     Demanda selecionarDemanda(List<Demanda> demandas, double orcamentoDisponivel);
8
9     // Nome amigável que exibo para o usuário
10    String getNomeEstrategia();
11
12    // Filtro comum às estratégias (DRY): só pedidos PENDENTES e com quantidade > 0
13    default List<Demanda> filtrarElegiveis(List<Demanda> demandas) {
14        List<Demanda> elegiveis = new ArrayList<>();
15        for (Demanda demanda : demandas) {
16            if (demanda.estaElegivel()) {
17                elegiveis.add(demanda);
18            }
19        }
20    }
```

```
20         return elegiveis;
21     }
22 }
```

Vocês já têm os `import` de `ArrayList` e `List`; falta só o método `default`.

Itens 14 a 16 · As três estratégias

X falta criar

Estrategia*.java

Todas seguem o mesmo molde: percorrem os elegíveis e guardam o "melhor até agora" segundo o seu critério.

EstrategiaFilaDeReceitas (ordem de chegada) TAREFA 3

1 Lógica

Atende quem chegou primeiro: o pedido de menor número.

2 Pseudocódigo

```
escolhida <- nada
para cada pedido elegível:
    se escolhida vazia ou pedido.numero < escolhida.numero: escolhida <- pedido
devolve escolhida
```

3 Código antigo (Tarefa 2)

Não existia: na Tarefa 2 quem escolhia era o usuário, pelo menu.

4 Como está agora (seu arquivo)

Ainda não existe no seu `tarefa-3/src`.

5 Como deve ficar

tarefa-3 · final

```
1 import java.util.List;
2
3 /**
4  * "Ordem de Chegada" no tema da fábrica: as receitas/pedidos são atendidos
5  * na ordem em que chegaram (FIFO), como a fila do balcão da farmácia.
6  */
7 public class EstrategiaFilaDeReceitas implements EstrategiaProducao {
8
9     @Override
10     public Demanda selecionarDemanda(List<Demanda> demandas, double orcamentoDisponivel) {
11         Demanda escolhida = null;
12         for (Demanda demanda : filtrarElegiveis(demandas)) {
13             // menor número = chegou antes
14             if (escolhida == null || demanda.getNumero() < escolhida.getNumero()) {
15                 escolhida = demanda;
16             }
17         }
18         return escolhida;
19     }
20
21     @Override
22     public String getNomeEstrategia() {
23         return "Fila de Receitas (ordem de chegada)";
24     }
25 }
```

EstrategiaSurtoEpidemico (maior pedido) TAREFA 3

1 Lógica

Num surto, prioriza o pedido com mais unidades. Não olha o caixa; se não couber, o gerenciador avisa.

2 Pseudocódigo

```
escolhida <- nada
para cada pedido elegível:
    se escolhida vazia ou pedido.quantidade > escolhida.quantidade: escolhida <- pedido
devolve escolhida
```

3 Código antigo (Tarefa 2)

Não existia na Tarefa 2.

4 Como está agora (seu arquivo)

Ainda não existe no seu `tarefa-3/src`.

5 Como deve ficar

tarefa-3 · final

```
1 import java.util.List;
2
3 /**
4  * "Maior Demanda" no tema: em um surto (dengue, gripe...) a prioridade é o
5  * pedido com MAIS unidades, porque atende mais pacientes de uma vez.
6  * Observação: esta estratégia não olha o orçamento. Se o maior pedido não
7  * couber no caixa, o gerenciador avisa e a fábrica precisa se ajustar.
8  */
9 public class EstrategiaSurtoEpidemico implements EstrategiaProducao {
10
11     @Override
12     public Demanda selecionarDemanda(List<Demanda> demandas, double orcamentoDisponivel) {
13         Demanda escolhida = null;
14         for (Demanda demanda : filtrarElegiveis(demandas)) {
15             if (escolhida == null || demanda.getQuantidade() > escolhida.getQuantidade()) {
16                 escolhida = demanda; // empate: fica o que chegou antes (o primeiro visto)
17             }
18         }
19         return escolhida;
20     }
21
22     @Override
23     public String getNomeEstrategia() {
24         return "Surto Epidêmico (maior pedido primeiro)";
25     }
26 }
```

EstrategiaFarmaciaPopular (máximo no orçamento) TAREFA 3

1 Lógica

Produz o máximo de unidades **sem estourar o caixa**: pula pedidos que não cabem e, entre os que cabem, fica com o maior. Empate: o mais barato.

2 Pseudocódigo

```
escolhida <- nada
para cada pedido elegível:
    se não cabe no caixa: pula
    se escolhida vazia ou (mais unidades, ou mesmas unidades e mais barato): escolhida <- pedido
devolve escolhida (nada = nenhum cabe)
```

3 Código antigo (Tarefa 2)

Não existia na Tarefa 2.

4 Como está agora (seu arquivo)

Ainda não existe no seu `tarefa-3/src`.

5 Como deve ficar

tarefa-3 · final

```
1 import java.util.List;
2
3 /**
4  * "Máximo de Produtos" no tema: como um programa de farmácia popular, a meta
5  * é entregar o MAIOR número de unidades possível SEM estourar o orçamento.
6  * Só considera pedidos cujo custo estimado cabe no caixa atual.
7  */
8 public class EstrategiaFarmaciaPopular implements EstrategiaProducao {
9
10     @Override
11     public Demanda selecionarDemanda(List<Demanda> demandas, double orcamentoDisponivel) {
12         Demanda escolhida = null;
13         for (Demanda demanda : filtrarElegiveis(demandas)) {
14             if (!demanda.cabeNoOrcamento(orcamentoDisponivel)) {
15                 continue; // inviável: estouraria o budget
16             }
17             if (escolhida == null || produzMaisOuCustaMenos(demanda, escolhida)) {
18                 escolhida = demanda;
19             }
20         }
21         return escolhida; // null = nenhum pedido cabe no orçamento
22     }
23
24     /** Critério principal: mais unidades. Desempate: menor custo. */
25     private boolean produzMaisOuCustaMenos(Demanda candidata, Demanda atual) {
26         if (candidata.getQuantidade() != atual.getQuantidade()) {
27             return candidata.getQuantidade() > atual.getQuantidade();
28         }
29         return candidata.calcularCustoEstimado() < atual.calcularCustoEstimado();
30     }
31
32     @Override
33     public String getNomeEstrategia() {
34         return "Farmácia Popular (máximo de unidades no orçamento)";
35     }
36 }
```

7. Etapa E · Juntar tudo (itens 17 a 21)

[↑ voltar ao sumário](#)

Quando esta etapa terminar, o projeto inteiro volta a compilar com o comando oficial.

Item 17 · Terminal

X falta criar `Terminal.java`

Cores, títulos e leitura segura **TAREFA 3**

1 Lógica

Na Tarefa 2, cada `case` do `Main` repetia a validação de entrada (`hasNextInt`, mensagem, `next()`). O `Terminal` junta isso num lugar só: títulos, mensagens coloridas (verde, amarelo, vermelho), dinheiro em R\$ e leitura que repete até o número ser válido. Classe `final` com construtor privado: só métodos `static`.

2 Pseudocódigo

```
cores só se houver terminal de verdade
titulo, secao, opcao, sucesso, aviso, erro, info: imprimem formatado
moeda(valor): "R$ 1.234,50"
lerInteiro(scanner, pergunta, min, max):
    repete: lê a linha ; se é inteiro entre min e max: devolve ; senão: erro
lerDecimal: igual, aceita vírgula ; lerTexto: lê uma linha
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 // Validação de entrada numérica
2 if (!scanner.hasNextInt()) {
3     System.out.println("Entrada inválida! Digite apenas números.");
4     scanner.next();
5     continue;
6 }
```

Esse trecho (e variações) aparecia várias vezes no `Main`.

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```
1 import java.util.Locale;
2 import java.util.Scanner;
3
4 /**
5  * Utilitários de console: cores ANSI, títulos, mensagens e leitura segura.
6  * Centralizar aqui evita repetir o mesmo "System.out.println + validação"
7  * em dezenas de lugares (DRY) e deixa o visual do menu consistente.
8  *
9  * Classe "final" com construtor privado: só tem métodos estáticos e não deve
10 * ser instanciada nem herdada.
11 */
12 public final class Terminal {
13
14     // Cores só quando há terminal de verdade (e o usuário não pediu NO_COLOR).
15     private static final boolean CORES = System.console() != null && System.getenv("NO_COLOR") == null;
16
17     public static final String RESET = codigo("\u001B[0m");
18     public static final String NEGRITO = codigo("\u001B[1m");
19     public static final String VERMELHO = codigo("\u001B[31m");
20     public static final String VERDE = codigo("\u001B[32m");
21     public static final String AMARELO = codigo("\u001B[33m");
```

```

22 public static final String AZUL = codigo("\u001B[34m");
23 public static final String CIANO = codigo("\u001B[36m");
24
25 public static final int LARGURA = 72;
26 private static final Locale BRASIL = Locale.forLanguageTag("pt-BR");
27
28 private Terminal() {
29 }
30
31 private static String codigo(String ansi) {
32     return CORES ? ansi : "";
33 }
34
35 // ----- desenho
36
37 public static String linha(char simbolo) {
38     StringBuilder sb = new StringBuilder();
39     for (int i = 0; i < LARGURA; i++) {
40         sb.append(simbolo);
41     }
42     return sb.toString();
43 }
44
45 public static void titulo(String texto) {
46     System.out.println();
47     System.out.println(CIANO + linha('=') + RESET);
48     System.out.println(NEGRITO + " " + texto.toUpperCase(BRASIL) + RESET);
49     System.out.println(CIANO + linha('=') + RESET);
50 }
51
52 public static void secao(String texto) {
53     System.out.println();
54     System.out.println(AZUL + linha('-') + RESET);
55     System.out.println(NEGRITO + "[" + texto.toUpperCase(BRASIL) + "]" + RESET);
56     System.out.println(AZUL + linha('-') + RESET);
57 }
58
59 public static void opcao(int numero, String texto) {
60     System.out.printf(" %s%d%s - %s%n", AMARELO, numero, RESET, texto);
61 }
62
63 // ----- mensagens
64
65 public static void sucesso(String msg) {
66     System.out.println(VERDE + " [OK] " + msg + RESET);
67 }
68
69 public static void aviso(String msg) {
70     System.out.println(AMARELO + " [!] " + msg + RESET);
71 }
72
73 public static void erro(String msg) {
74     System.out.println(VERMELHO + " [X] " + msg + RESET);
75 }
76
77 public static void info(String msg) {
78     System.out.println(" " + msg);
79 }
80
81 /** Formata dinheiro no padrão brasileiro: R$ 1.234,50 */
82 public static String moeda(double valor) {
83     return String.format(BRASIL, "R$ %,2f", valor);
84 }
85
86 // ----- leitura segura
87
88 /**
89  * Lê um inteiro entre min e max, repetindo até a entrada ser válida.
90  * Se a entrada acabar (ex.: teste com arquivo), devolve 0 = voltar/sair.
91  */
92 public static int lerInteiro(Scanner scanner, String prompt, int min, int max) {
93     while (true) {
94         System.out.print(prompt);
95         if (!scanner.hasNextLine()) {

```

```

96         System.out.println();
97         return 0;
98     }
99     String texto = scanner.nextLine().trim();
100     try {
101         int valor = Integer.parseInt(texto);
102         if (valor >= min && valor <= max) {
103             return valor;
104         }
105     } catch (NumberFormatException e) {
106         // cai na mensagem de erro abaixo
107     }
108     erro(String.format("Digite um número inteiro entre %d e %d.", min, max));
109 }
110 }
111
112 /** Lê um número decimal >= 0 (aceita vírgula ou ponto). EOF devolve 0. */
113 public static double lerDecimal(Scanner scanner, String prompt) {
114     while (true) {
115         System.out.print(prompt);
116         if (!scanner.hasNextLine()) {
117             System.out.println();
118             return 0;
119         }
120         String texto = scanner.nextLine().trim().replace(',', '.');
121         try {
122             double valor = Double.parseDouble(texto);
123             if (valor >= 0) {
124                 return valor;
125             }
126         } catch (NumberFormatException e) {
127             // cai na mensagem de erro abaixo
128         }
129         erro("Digite um número maior ou igual a zero (ex.: 25,5).");
130     }
131 }
132
133 public static String lerTexto(Scanner scanner, String prompt) {
134     System.out.print(prompt);
135     return scanner.hasNextLine() ? scanner.nextLine().trim() : "";
136 }
137 }

```

Item 18 · GerenciadorProducao

X ainda da Tarefa 2

GerenciadorProducao.java

É o arquivo que mais muda. Continua coordenando pedidos, máquinas, estoque, armazém e caixa, mas: é o **contexto** do Strategy, valida tudo antes de fabricar, aplica desgaste, repara máquinas e faz a auditoria. Menu e montagem da fábrica saem dele (itens 19 e 20).

Atributos e construtor MUDA

1 Lógica

As listas viram `final` (a lista é sempre a mesma; o conteúdo muda). Entram o cenário, a estratégia atual e o contador de lotes. O construtor recebe o cenário (de onde vem o budget) e a estratégia inicial.

2 Pseudocódigo

```
constantes: CUSTO_REPARO = 150 ; TAXA_REJEICAO_ALERTA = 0.30
listas: demandas, armazem, maquinas
materiaPrima, cenario, estrategiaAtual, budget, contadorLotes
construtor(materiaPrima, cenario, estrategia): budget <- budget do cenário
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 public class GerenciadorProducao {
2
3     // Atributos
4     private ArrayList<Demanda> demandas;
5     private ArrayList<Produto> produtosFabricados;
6     private ArrayList<Maquina> maquinas;
7     private MateriaPrima materiaPrima;
8     private double budget;
9
10    // Construtor
11    public GerenciadorProducao(MateriaPrima materiaPrima, double budgetInicial) {
12        this.demandas = new ArrayList<>();
13        this.produtosFabricados = new ArrayList<>();
14        this.maquinas = new ArrayList<>();
15        this.materiaPrima = materiaPrima;
16        this.budget = budgetInicial;
17    }
```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```
1 public class GerenciadorProducao {
2
3     private static final double CUSTO_REPARO = 150.0;
4     private static final double TAXA_REJEICAO_ALERTA = 0.30;
5
6     private final List<Demanda> demandas = new ArrayList<>();
7     private final List<Produto> armazem = new ArrayList<>();
8     private final List<Maquina> maquinas = new ArrayList<>();
9     private final MateriaPrima materiaPrima;
10    private final Cenario cenario;
11    private EstrategiaProducao estrategiaAtual;
12    private double budget;
13    private int contadorLotes = 0;
14
15    public GerenciadorProducao(MateriaPrima materiaPrima, Cenario cenario,
16                               EstrategiaProducao estrategiaInicial) {
```

```

17     this.materiaPrima = materiaPrima;
18     this.cenario = cenario;
19     this.estrategiaAtual = estrategiaInicial;
20     this.budget = cenario.getBudgetInicial();
21 }

```

Strategy: trocar e usar a estratégia TAREFA 3

1 Lógica

`setEstrategia` troca o critério com o programa rodando. `executarProximaProducao` pergunta à estratégia atual qual pedido fazer e fabrica. A lista vai como somente-leitura, para a estratégia não mexer nos pedidos. O gerenciador só conhece a **interface**. `fabricarDemanda` continua existindo para o modo manual, agora pelo número do pedido.

2 Pseudocódigo

```

setEstrategia(nova): se nula: erro ; senão estrategiaAtual <- nova
executarProximaProducao:
    escolhida <- estrategiaAtual.selecionarDemanda(pedidos só-leitura, budget)
    se nada: aviso ; senão fabricar(escolhida)
fabricarDemanda(numero): busca o pedido ; fabricar(pedido)

```

3 Código antigo (Tarefa 2)

Não existia na Tarefa 2.

4 Como está agora (seu arquivo)

Ainda não existe no seu `tarefa-3/src`.

5 Como deve ficar

tarefa-3 · final

```

1  /** Troca o critério de produção em tempo de execução. */
2  public void setEstrategia(EstrategiaProducao novaEstrategia) {
3      if (novaEstrategia == null) {
4          Terminal.erro("Estratégia inválida.");
5          return;
6      }
7      this.estrategiaAtual = novaEstrategia;
8      Terminal.sucesso("Estratégia alterada para: " + novaEstrategia.getNomeEstrategia());
9  }
10
11 /** Pergunta à estratégia atual qual pedido produzir e fabrica. */
12 public boolean executarProximaProducao() {
13     // Lista somente-leitura: a estratégia não consegue adicionar/remover pedidos.
14     Demanda escolhida = estrategiaAtual.selecionarDemanda(
15         Collections.unmodifiableList(demandas), budget);
16
17     if (escolhida == null) {
18         Terminal.aviso("A estratégia \"" + estrategiaAtual.getNomeEstrategia()
19             + "\" não encontrou pedido elegível (pendente e viável).");
20         return false;
21     }
22     Terminal.info("Estratégia \"" + estrategiaAtual.getNomeEstrategia() + "\" escolheu: " +
escolhida);
23     return fabricar(escolhida);
24 }
25
26 /** Fabricação manual de um pedido específico (ignora a estratégia). */
27 public boolean fabricarDemanda(int numeroPedido) {
28     Demanda demanda = buscarDemanda(numeroPedido);
29     if (demanda == null) {
30         Terminal.erro("Pedido #" + numeroPedido + " não encontrado.");
31         return false;
32     }

```

```

33     return fabricar(demanda);
34 }

```

Pedidos: registrar, atualizar, cancelar MUDA

1 Lógica

O pedido é achado pelo **número**, não pela posição na lista. `registrarDemanda` monta a `Demanda` já com o custo da linha. Atualizar e cancelar conferem se o status permite antes de agir.

2 Pseudocódigo

```

registrarDemanda(tipo, nome, qtd): cria a Demanda com o custo da linha ; adiciona
atualizarDemanda(numero, qtd): busca ; se não pode ser atualizada: erro ; senão atualiza
cancelarDemanda(numero): busca ; se status não permite CANCELADA: erro ; senão cancela
buscarDemanda(numero): percorre a lista ; devolve o pedido ou null

```

3 Código antigo (Tarefa 2)

tarefa-2

```

1  // Métodos de Demanda
2  public void registrarDemanda(Demanda demanda) {
3      demandas.add(demanda);
4  }
5
6  public void atualizarDemanda(int indice, int novaQuantidade) {
7      if (novaQuantidade < 0) {
8          System.out.println("Erro: A quantidade da demanda não pode ser negativa.");
9          return;
10     }
11     demandas.get(indice).atualizarQuantidade(novaQuantidade);
12     System.out.println("Demanda de \"" + demandas.get(indice).getTipoProduto() + "\" atualizada para "
+ novaQuantidade + " unidades.");
13 }

```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```

1  public Demanda registrarDemanda(TipoMedicamento tipo, String medicamento, int quantidade) {
2      Demanda demanda = new Demanda(tipo, medicamento, quantidade, calcularCustoLinhaPorUnidade());
3      demandas.add(demanda);
4      return demanda;
5  }
6
7  public void atualizarDemanda(int numeroPedido, int novaQuantidade) {
8      Demanda demanda = buscarDemanda(numeroPedido);
9      if (demanda == null) {
10         Terminal.erro("Pedido #" + numeroPedido + " não encontrado.");
11     } else if (!demanda.podeSerAtualizada()) {
12         Terminal.erro("Pedido #" + numeroPedido + " está " + demanda.getStatus().getDescricao()
13             + " e não pode ser alterado.");
14     } else {
15         demanda.atualizarQuantidade(novaQuantidade);
16         Terminal.sucesso("Pedido #" + numeroPedido + " agora tem " + novaQuantidade
17             + " unidade(s) (" + demanda.getStatus().getDescricao() + ").");
18     }
19 }
20
21 public void cancelarDemanda(int numeroPedido) {
22     Demanda demanda = buscarDemanda(numeroPedido);
23     if (demanda == null) {
24         Terminal.erro("Pedido #" + numeroPedido + " não encontrado.");
25     } else if (!demanda.getStatus().mudaPara(StatusDemanda.CANCELADA)) {
26         Terminal.erro("Só pedidos pendentes podem ser cancelados (status atual: "

```

```

27         + demanda.getStatus().getDescricao() + ").");
28     } else {
29         demanda.cancelar();
30         Terminal.sucesso("Pedido #" + numeroPedido + " cancelado.");
31     }
32 }
33
34 private Demanda buscarDemanda(int numeroPedido) {
35     for (Demanda demanda : demandas) {
36         if (demanda.getNumero() == numeroPedido) {
37             return demanda;
38         }
39     }
40     return null;
41 }

```

fabricar: validar tudo, depois produzir MUDA

1 Lógica

O antigo `fabricarDemanda(indice)` tinha `if` por índice para consumo e classe. Agora o tipo do pedido resolve isso. Primeiro valida **tudo** (status, máquina quebrada, capacidade, insumo, caixa); só então muda o estado. Cada unidade é criada por `tipo.criarProduto`, passa pelas máquinas e vai para o armazém.

2 Pseudocódigo

```

fabricar(pedido):
    se não elegível / máquina quebrada / qtd > capacidade / falta insumo / falta caixa: erro, sai
    pedido.iniciarProducao ; consome insumo ; debita custo ; gera código do lote
    liga a linha
    repete qtd vezes:
        produto <- tipo.criarProduto(nome, lote) ; iniciarProcessamento
        passa por cada máquina ; guarda no armazém ; conta se aprovado
    desliga a linha ; pedido.concluir
    imprime relatório ; registra desgaste

```

3 Código antigo (Tarefa 2)

tarefa-2

```

1  // Produção
2  public void fabricarDemanda(int indiceDemanda) {
3      Demanda demanda = demandas.get(indiceDemanda);
4
5      // Verificar se a demanda já foi atendida
6      if (demanda.isAtendida()) {
7          System.out.println("Aviso: Esta demanda já foi atendida anteriormente.");
8          return;
9      }
10
11     int quantidade = demanda.getQuantidadeProdutos();
12     if (quantidade <= 0) {
13         System.out.println("Aviso: A quantidade da demanda é zero. Atualize a demanda antes de
fabricar.");
14         return;
15     }
16
17     // Definir consumo de matéria-prima por unidade e tipo de produto
18     double consumoPorUnidade;
19     if (indiceDemanda == 0) {
20         consumoPorUnidade = 3.0;
21     } else if (indiceDemanda == 1) {
22         consumoPorUnidade = 2.0;
23     } else {
24         consumoPorUnidade = 1.0;
25     }
26
27     // Verificar disponibilidade de matéria-prima necessária

```

```

28     double materiaPrimaNecessaria = demanda.calcularMateriaPrimaNecessaria(consumoPorUnidade);
29     if (!materiaPrima.verificarDisponibilidade(materiaPrimaNecessaria)) {
30         System.out.printf("Erro: Matéria-prima insuficiente! Necessário: %.2f %s | Disponível: %.2f
31         %s\n",
32             materiaPrimaNecessaria, materiaPrima.getUnidade(), materiaPrima.getQuantidade(),
33             materiaPrima.getUnidade());
34         System.out.println("Compre mais matéria-prima antes de iniciar a produção.");
35         return;
36     }
37     // Verificar budget para debitar custo de operação das máquinas
38     double custoOperacional = calcularCustoProducao(quantidade);
39     if (budget < custoOperacional) {
40         System.out.printf("Erro: Budget insuficiente para pagar a operação das máquinas! Necessário:
41         R$ %.2f | Disponível: R$ %.2f\n",
42             custoOperacional, budget);
43         return;
44     }
45     // Consumir matéria-prima e debitar budget
46     materiaPrima.consumir(materiaPrimaNecessaria);
47     budget -= custoOperacional;
48     // Ligar máquinas para a produção
49     for (Maquina maquina : maquinas) {
50         maquina.ligar();
51     }
52
53     int aprovados = 0;
54     int rejeitados = 0;
55
56     // (Processamento -> Embalagem -> Inspeção)
57     for (int i = 0; i < quantidade; i++) {
58         Produto produto;
59         int idProduto = Produto.getTotalProdutosFabricados() + 1;
60
61         if (indiceDemanda == 0) {
62             produto = new MedicamentoControlado(idProduto, "Medicamento Alta Qualidade #" + idProduto,
63             "Aguardando", consumoPorUnidade);
64         } else if (indiceDemanda == 1) {
65             produto = new MedicamentoContinuo(idProduto, "Medicamento Média Qualidade #" + idProduto,
66             "Aguardando", consumoPorUnidade);
67         } else {
68             produto = new MedicamentoGenerico(idProduto, "Medicamento Baixa Qualidade #" + idProduto,
69             "Aguardando", consumoPorUnidade);
70         }
71
72         produto.processar();
73
74         // Passagem pelas máquinas da linha de produção
75         for (Maquina maquina : maquinas) {
76             maquina.processar(produto);
77         }
78
79         // Adicionar ao armazém
80         produtosFabricados.add(produto);
81
82         if ("Aprovado".equalsIgnoreCase(produto.getStatus())) {
83             aprovados++;
84         } else {
85             rejeitados++;
86         }
87     }
88
89     // Desligar máquinas após a produção
90     for (Maquina m : maquinas) {
91         m.desligar();
92     }
93
94     // Marcar demanda como atendida
95     demanda.atender();
96
97     // Exibir relatório da fabricação
98     System.out.println("\n=====");

```

```

96     System.out.println("          RELATÓRIO DE FABRICAÇÃO");
97     System.out.println("=====");
98     System.out.println("Demanda atendida: " + demanda.getTipoProduto());
99     System.out.println("Total produzido: " + quantidade + " unidades");
100    System.out.println(" - Aprovados na Inspeção (Anvisa): " + aprovados + " unidades");
101    System.out.println(" - Rejeitados no Controle: " + rejeitados + " unidades");
102    System.out.printf("Matéria-prima consumida: %.2f %s (Restante: %.2f %s)\n",
103                      materiaPrimaNecessaria, materiaPrima.getUnidade(), materiaPrima.getQuantidade(),
materiaPrima.getUnidade());
104    System.out.printf("Custo operacional debitado: R$ %.2f (Budget atual: R$ %.2f)\n",
105                      custoOperacional, budget);
106    System.out.println("=====\\n");
107 }

```

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```

1     public void fabricarDemanda(int indiceDemanda) {
2         Demanda demanda = demandas.get(indiceDemanda);
3
4         // Verificar se a demanda já foi atendida
5         if (demanda.isAtendida()) {
6             System.out.println("Aviso: Esta demanda já foi atendida anteriormente.");
7             return;
8         }
9
10        int quantidade = demanda.getQuantidadeProdutos();
11        if (quantidade <= 0) {
12            System.out.println("Aviso: A quantidade da demanda é zero. Atualize a demanda antes de
fabricar.");
13            return;
14        }
15
16        // Definir consumo de matéria-prima por unidade e tipo de produto
17        double consumoPorUnidade;
18        if (indiceDemanda == 0) {
19            consumoPorUnidade = 3.0;
20        } else if (indiceDemanda == 1) {
21            consumoPorUnidade = 2.0;
22        } else {
23            consumoPorUnidade = 1.0;
24        }
25
26        // Verificar disponibilidade de matéria-prima necessária
27        double materiaPrimaNecessaria = demanda.calcularMateriaPrimaNecessaria(consumoPorUnidade);
28        if (!materiaPrima.verificarDisponibilidade(materiaPrimaNecessaria)) {
29            System.out.printf("Erro: Matéria-prima insuficiente! Necessário: %.2f %s | Disponível: %.2f
%s\\n",
30                              materiaPrimaNecessaria, materiaPrima.getUnidade(), materiaPrima.getQuantidade(),
materiaPrima.getUnidade());
31            System.out.println("Compre mais matéria-prima antes de iniciar a produção.");
32            return;
33        }
34
35        // Verificar budget para debitar custo de operação das máquinas
36        double custoOperacional = calcularCustoProducao(quantidade);
37        if (budget < custoOperacional) {
38            System.out.printf("Erro: Budget insuficiente para pagar a operação das máquinas! Necessário:
R$ %.2f | Disponível: R$ %.2f\\n",
39                              custoOperacional, budget);
40            return;
41        }
42
43        // Consumir matéria-prima e debitar budget
44        materiaPrima.consumir(materiaPrimaNecessaria);
45        budget -= custoOperacional;
46
47        // Ligar máquinas para a produção
48        for (Máquina maquina : maquinas) {
49            maquina.ligar();
50        }
51    }

```

```

52     int aprovados = 0;
53     int rejeitados = 0;
54
55     // (Processamento -> Embalagem -> Inspeção)
56     for (int i = 0; i < quantidade; i++) {
57         Produto produto;
58         int idProduto = Produto.getTotalProdutosFabricados() + 1;
59
60         if (indiceDemanda == 0) {
61             produto = new MedicamentoControlado(idProduto, "Medicamento Controlado #" + idProduto,
"Aguardando", consumoPorUnidade);
62         } else if (indiceDemanda == 1) {
63             produto = new MedicamentoContínuo(idProduto, "Medicamento Contínuo #" + idProduto,
"Aguardando", consumoPorUnidade);
64         } else {
65             produto = new MedicamentoGenerico(idProduto, "Medicamento Genérico #" + idProduto,
"Aguardando", consumoPorUnidade);
66         }
67
68         produto.processar();
69
70         // Passagem pelas máquinas da linha de produção
71         for (Maquina maquina : maquinas) {
72             maquina.processar(produto);
73         }
74
75         // Adicionar ao armazém
76         produtosFabricados.add(produto);
77
78         if ("Aprovado".equalsIgnoreCase(produto.getStatus())) {
79             aprovados++;
80         } else {
81             rejeitados++;
82         }
83     }
84
85     // Desligar máquinas após a produção
86     for (Maquina m : maquinas) {
87         m.desligar();
88     }
89
90     // Marcar demanda como atendida
91     demanda.atender();
92
93     // Exibir relatório da fabricação
94     System.out.println("\n=====");
95     System.out.println("          RELATÓRIO DE FABRICAÇÃO");
96     System.out.println("=====");
97     System.out.println("Demanda atendida: " + demanda.getTipoProduto());
98     System.out.println("Total produzido: " + quantidade + " unidades");
99     System.out.println(" - Aprovados na Inspeção (Anvisa): " + aprovados + " unidades");
100    System.out.println(" - Rejeitados no Controle: " + rejeitados + " unidades");
101    System.out.printf("Matéria-prima consumida: %.2f %s (Restante: %.2f %s)\n",
102        materiaPrimaNecessaria, materiaPrima.getUnidade(), materiaPrima.getQuantidade(),
materiaPrima.getUnidade());
103    System.out.printf("Custo operacional debitado: R$ %.2f (Budget atual: R$ %.2f)\n",
104        custoOperacional, budget);
105    System.out.println("=====");
106 }

```

5 Como deve ficar

tarefa-3 · final

```

1  /**
2   * Fluxo único de fabricação (usado pela estratégia e pelo modo manual).
3   * Primeiro valida TUDO; só depois muda estado, estoque e caixa. Assim um
4   * pedido nunca fica "EM_PRODUCAO" pela metade.
5   */
6  private boolean fabricar(Demanda demanda) {
7      if (!demanda.estaElegivel()) {
8          Terminal.avisar("Pedido #" + demanda.getNumero() + " não pode ser fabricado (status "
9              + demanda.getStatus().getDescricao() + ", " + demanda.getQuantidade() + " un.).");

```

```

10         return false;
11     }
12     Maquina quebrada = buscarMaquinaQuebrada();
13     if (quebrada != null) {
14         Terminal.erro("Linha parada: \"" + quebrada.getNome()
15             + "\" está quebrada. Repare-a em Auditoria e Manutenção.");
16         return false;
17     }
18     int quantidade = demanda.getQuantidade();
19     int capacidade = calcularCapacidadeDaLinha();
20     if (quantidade > capacidade) {
21         Terminal.erro("0 pedido tem " + quantidade + " un., mas a linha processa no máximo "
22             + capacidade + " por lote. Divida o pedido.");
23         return false;
24     }
25     double consumo = demanda.calcularConsumoEstimado();
26     if (!materiaPrima.verificarDisponibilidade(consumo)) {
27         Terminal.erro(String.format("Princípio ativo insuficiente: precisa %.1f %s, há %.1f %s. "
28             + "Compre matéria-prima ou cancele o pedido.",
29             consumo, materiaPrima.getUnidade(), materiaPrima.getQuantidade(),
30             materiaPrima.getUnidade()));
31         return false;
32     }
33     double custo = demanda.calcularCustoEstimado();
34     if (custo > budget) {
35         Terminal.erro("Orçamento insuficiente: a operação custa " + Terminal.moeda(custo)
36             + " e o caixa tem " + Terminal.moeda(budget) + ".");
37         return false;
38     }
39     // ---- validado: agora sim altera o estado da fábrica
40     demanda.iniciarProducao();
41     materiaPrima.consumir(consumo);
42     budget -= custo;
43     String lote = gerarCodigoLote(demanda.getTipo());
44
45     ligarLinha();
46     int aprovados = 0;
47     for (int i = 0; i < quantidade; i++) {
48         Produto produto = demanda.getTipo().criarProduto(demanda.getMedicamento(), lote);
49         produto.iniciarProcessamento();
50         for (Maquina maquina : maquinas) { // Processamento -> Embalagem -> Inspeção
51             maquina.processar(produto);
52         }
53         armazem.add(produto);
54         if (produto.foiAprovado()) {
55             aprovados++;
56         }
57     }
58     desligarLinha();
59     demanda.concluir();
60
61     imprimirRelatorioFabricacao(demanda, lote, aprovados, consumo, custo);
62     registrarDesgasteDaLinha();
63     return true;
64 }

```

Linha, desgaste, relatório e lote TAREFA 3

1 Lógica

Pedaços que eram código solto dentro do `fabricarDemanda` viram métodos pequenos com nome. Novo: depois de cada lote, cada máquina sofre desgaste e o sistema avisa se quebrou ou se precisa de manutenção. O lote ganha um código (L001-CTR).

2 Pseudocódigo

ligarLinha / desligarLinha: liga/desliga cada máquina
registrarDesgasteDaLinha: para cada máquina: registrarCicloDeUso ; avisa se quebrou ou precisa de manutenção
imprimirRelatorioFabricacao: aprovados, rejeitados, insumo, custo, caixa + mensagem conforme a taxa de rejeição
gerarCodigoLote(tipo): contador + 1 ; "L" + 3 dígitos + "-" + sigla

3 Código antigo (Tarefa 2)

Ligar, desligar e o relatório existiam, mas escritos dentro do `fabricarDemanda` (bloco anterior).

4 Como está agora (seu arquivo)

Ainda não existe no seu `tarefa-3/src`.

5 Como deve ficar

tarefa-3 · final

```
1  private void ligarLinha() {
2      for (Maquina maquina : maquinas) {
3          maquina.ligar();
4      }
5  }
6
7  private void desligarLinha() {
8      for (Maquina maquina : maquinas) {
9          maquina.desligar();
10     }
11 }
12
13 /** Um ciclo de fabricação = um uso de cada máquina = um desgaste. */
14 private void registrarDesgasteDaLinha() {
15     Terminal.info("Desgaste da linha neste lote:");
16     for (Maquina maquina : maquinas) {
17         double perda = maquina.registrarCicloDeUso();
18         String texto = String.format("  %-22s saúde %5.1f%% (-%.1f)",
19             maquina.getNome(), maquina.getSaude(), perda);
20         if (maquina.estaQuebrada()) {
21             Terminal.erro(texto + " QUEBROU! A linha fica parada até o reparo.");
22         } else if (maquina.precisaManutencao()) {
23             Terminal.aviso(texto + " manutenção recomendada.");
24         } else {
25             Terminal.info(texto);
26         }
27     }
28 }
29
30 private void imprimirRelatorioFabricacao(Demanda demanda, String lote, int aprovados,
31     double consumo, double custo) {
32     int total = demanda.getQuantidade();
33     int rejeitados = total - aprovados;
34     Terminal.titulo("Relatório de fabricação - lote " + lote);
35     Terminal.info(demanda.toString());
36     Terminal.info(String.format("Aprovados na inspeção: %d | Rejeitados: %d", aprovados, rejeitados));
37     Terminal.info(String.format("Princípio ativo consumido: %.1f %s (restam %.1f %s)",
38         consumo, materiaPrima.getUnidade(), materiaPrima.getQuantidade(),
39         materiaPrima.getUnidade()));
40     Terminal.info("Custo operacional: " + Terminal.moeda(custo) + " | Caixa: " +
41         Terminal.moeda(budget));
42
43     // Toque temático: mensagens de acordo com o resultado do lote
44     if (rejeitados == 0) {
45         Terminal.sucesso("Lote " + lote + " 100% aprovado. Liberado para distribuição!");
46     } else if ((double) rejeitados / total >= TAXA_REJEICAO_ALERTA) {
47         Terminal.erro("Alerta de qualidade: " + rejeitados + " unidade(s) reprovada(s). "
48             + "A vigilância sanitária vai querer ver esse lote!");
49     } else {
50         Terminal.aviso(rejeitados + " unidade(s) seguem para reanálise no laboratório.");
51     }
52 }
```

```

50     }
51
52     private String gerarCodigoLote(TipoMedicamento tipo) {
53         contadorLotes++;
54         return String.format("L%03d-%s", contadorLotes, tipo.getSigla());
55     }

```

Máquinas: custo, capacidade, reparo MUDA

1 Lógica

calcularCustoProducao vira calcularCustoLinhaPorUnidade (o pedido multiplica pela quantidade). Novos: a capacidade da linha é a da máquina mais lenta; achar máquina quebrada; reparar (custa R\$ 150); contar as que funcionam.

2 Pseudocódigo

```

adicionarMaquina(m): adiciona
calcularCustoLinhaPorUnidade: soma o custo de cada máquina
calcularCapacidadeDaLinha: menor capacidade entre as máquinas
buscarMaquinaQuebrada: primeira quebrada ou null
repararMaquina(i): se índice inválido / saúde cheia / sem caixa: aviso ; senão paga 150 e repara
contarMaquinasOperantes: quantas não estão quebradas

```

3 Código antigo (Tarefa 2)

tarefa-2

```

1  // Métodos de Máquinas
2  public void adicionarMaquina(Maquina maquina) {
3      maquinas.add(maquina);
4  }
5
6  // Método privado
7  private double calcularCustoProducao(int quantidadeProdutos) {
8      double custoPorProduto = 0;
9      for (Maquina maquina : maquinas) {
10         custoPorProduto += maquina.getCustoOperacao();
11     }
12     return custoPorProduto * quantidadeProdutos;
13 }

```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```

1  public void adicionarMaquina(Maquina maquina) {
2      maquinas.add(maquina);
3  }
4
5  /** Soma o custo de todas as máquinas: quanto custa passar 1 unidade pela linha. */
6  private double calcularCustoLinhaPorUnidade() {
7      double custo = 0;
8      for (Maquina maquina : maquinas) {
9          custo += maquina.getCustoOperacao();
10     }
11     return custo;
12 }
13
14 /** A linha é tão rápida quanto sua máquina mais lenta (menor capacidade). */
15 private int calcularCapacidadeDaLinha() {
16     int capacidade = Integer.MAX_VALUE;
17     for (Maquina maquina : maquinas) {
18         capacidade = Math.min(capacidade, maquina.getCapacidadeMaxima());
19     }

```

```

20     return capacidade;
21 }
22
23 private Maquina buscarMaquinaQuebrada() {
24     for (Maquina maquina : maquinas) {
25         if (maquina.estaQuebrada()) {
26             return maquina;
27         }
28     }
29     return null;
30 }
31
32 public void repararMaquina(int indice) {
33     if (indice < 0 || indice >= maquinas.size()) {
34         Terminal.erro("Máquina inexistente.");
35         return;
36     }
37     Maquina maquina = maquinas.get(indice);
38     if (maquina.getSaude() >= Maquina.SAUDE_MAXIMA) {
39         Terminal.avisos("\n" + maquina.getNome() + "\n já está com saúde máxima.");
40         return;
41     }
42     if (budget < CUSTO_REPARO) {
43         Terminal.erro("O reparo custa " + Terminal.moeda(CUSTO_REPARO) + " e o caixa tem "
44             + Terminal.moeda(budget) + ".");
45         return;
46     }
47     budget -= CUSTO_REPARO;
48     maquina.reparar();
49     Terminal.sucesso("\n" + maquina.getNome() + "\n revisada: saúde 100%. Custo "
50         + Terminal.moeda(CUSTO_REPARO) + ".");
51 }
52
53 public int contarMaquinasOperantes() {
54     int operantes = 0;
55     for (Maquina maquina : maquinas) {
56         if (!maquina.estaQuebrada()) {
57             operantes++;
58         }
59     }
60     return operantes;
61 }

```

Comprar matéria-prima MUDA

1 Lógica

Mesma lógica da Tarefa 2 (valida quantidade e caixa, debita, adiciona ao estoque). Só as mensagens passam pelo `Terminal`.

2 Pseudocódigo

```

comprarMateriaPrima(qtd):
    se qtd <= 0: erro
    custo <- qtd * preço ; se custo > budget: erro
    budget - custo ; estoque + qtd

```

3 Código antigo (Tarefa 2)

tarefa-2

```

1 // Matéria-Prima
2 public void comprarMateriaPrima(double quantidade) {
3     if (!(quantidade > 0)) {
4         System.out.println("Erro: A quantidade a ser comprada deve ser maior que zero.");
5         return;
6     }
7
8     // Calcular custo total da matéria-prima (quantidade * custoPorUnidade)
9     double custoTotal = quantidade * materiaPrima.getCustoPorUnidade();

```

```

10
11     // Validar budget disponível
12     if (custoTotal > budget) {
13         System.out.printf("Erro: Budget insuficiente! Custo da compra: R$ %.2f | Budget disponível: R$
%.2f\n",
14             custoTotal, budget);
15         return;
16     }
17
18     // Debitar do budget e adicionar ao estoque
19     budget -= custoTotal;
20     materiaPrima.adicionarEstoque(quantidade);
21     System.out.printf("Compra de %.2f %s realizada com sucesso! Custo: R$ %.2f | Novo Budget: R$
%.2f\n",
22         quantidade, materiaPrima.getUnidade(), custoTotal, budget);
23 }

```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```

1     public void comprarMateriaPrima(double quantidade) {
2         if (quantidade <= 0) {
3             Terminal.erro("A quantidade deve ser maior que zero.");
4             return;
5         }
6         double custo = quantidade * materiaPrima.getCustoPorUnidade();
7         if (custo > budget) {
8             Terminal.erro("Compra de " + Terminal.moeda(custo) + " excede o caixa ("
9                 + Terminal.moeda(budget) + ").");
10            return;
11        }
12        budget -= custo;
13        materiaPrima.adicionarEstoque(quantidade);
14        Terminal.sucesso(String.format("Caminhão descarregado: +%.1f %s por %s. Caixa: %s",
15            quantidade, materiaPrima.getUnidade(), Terminal.moeda(custo), Terminal.moeda(budget)));
16    }

```

Consultas: pedidos, armazém, estoque MUDA

1 Lógica

`exibirDemandas` mostra número, tipo, status (do enum), custo e insumo. O armazém passa a ser mostrado **por lote** (o enunciado pede visualização do armazém), com aprovados, rejeitados e risco médio. `exibirBudget` sai: o cabeçalho do menu já mostra o caixa.

2 Pseudocódigo

```

exibirDemandas: tabela com uma linha por pedido
exibirArmazem:
    agrupa os produtos por lote (mapa lote -> lista)
    para cada lote: conta aprovados, em risco e soma o risco
    imprime uma linha por lote + total
exibirEstoqueMateriaPrima: nome, quantidade, preço

```

3 Código antigo (Tarefa 2)

tarefa-2

```

1     public void exibirArmazem() {
2         System.out.println("\n=====");
3         System.out.println("          PRODUTOS NO ARMAZÉM");
4         System.out.println("=====");
5         if (produtosFabricados.isEmpty()) {
6             System.out.println("0 armazém está vazio no momento.");

```

```

7         } else {
8             for (Produto produto : produtosFabricados) {
9                 System.out.printf("ID: %-3d | Nome: %-32s | Tipo: %-32s | Qualidade: %.1f | Falha
Acumulada: %.2f | Status: %s\n",
10                     produto.getId(),
11                     produto.getNome(),
12                     produto.getTipo(),
13                     produto.getQualidade(),
14                     produto.getProbabilidadeFalhaAcumulada(),
15                     produto.getStatus());
16             }
17             System.out.println("Total no armazém: " + produtosFabricados.size() + " itens (Total geral
fabricado: " + Produto.getTotalProdutosFabricados() + ")");
18         }
19         System.out.println("=====\n");
20     }
21
22     public void exibirEstoqueMateriaPrima() {
23         System.out.println("\n=====");
24         System.out.println("          ESTOQUE DE MATÉRIA-PRIMA");
25         System.out.println("=====");
26         System.out.printf("ID: %d | Nome: %s\n", materiaPrima.getId(), materiaPrima.getNome());
27         System.out.printf("Quantidade disponível: %.2f %s\n", materiaPrima.getQuantidade(),
materiaPrima.getUnidade());
28         System.out.printf("Custo por unidade: R$ %.2f / %s\n", materiaPrima.getCustoPorUnidade(),
materiaPrima.getUnidade());
29         System.out.println("=====\n");
30     }
31
32     // Prints e Consultas
33     public void exibirBudget() {
34         System.out.printf("BUDGET ATUAL: R$ %.2f\n", budget);
35     }

```

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```

1     // Produção
2     public void exibirDemandas() {
3         System.out.println("\n=====");
4         System.out.println("          DEMANDAS REGISTRADAS");
5         System.out.println("=====");
6
7         if (demandas.isEmpty()) {
8             System.out.println("Nenhuma demanda registrada no momento.");
9             return;
10        }
11
12        for (int i = 0; i < demandas.size(); i++) {
13            Demanda demanda = demandas.get(i);
14            System.out.printf("Demanda %d: Tipo: %s | Quantidade: %d | Atendida: %s\n",
15                i + 1, demanda.getTipoProduto(), demanda.getQuantidadeProdutos(), demanda.isAtendida()
? "Sim" : "Não");
16        }
17    }
18
19    public void exibirArmazem() {
20        System.out.println("\n=====");
21        System.out.println("          PRODUTOS NO ARMAZÉM");
22        System.out.println("=====");
23        if (produtosFabricados.isEmpty()) {
24            System.out.println("0 armazém está vazio no momento.");
25        } else {
26            for (Produto produto : produtosFabricados) {
27                System.out.printf("ID: %-3d | Nome: %-32s | Tipo: %-32s | Qualidade: %.1f | Falha
Acumulada: %.2f | Status: %s\n",
28                    produto.getId(),
29                    produto.getNome(),
30                    produto.getTipo(),
31                    produto.getQualidade(),
32                    produto.getProbabilidadeFalhaAcumulada(),
33                    produto.getStatus());

```

```

34         }
35         System.out.println("Total no armazém: " + produtosFabricados.size() + " itens (Total geral
fabricado: " + Produto.getTotalProdutosFabricados() + ")");
36     }
37     System.out.println("=====\n");
38 }
39
40 public void exibirEstoqueMateriaPrima() {
41     System.out.println("\n=====");
42     System.out.println("          ESTOQUE DE MATÉRIA-PRIMA");
43     System.out.println("=====");
44     System.out.printf("ID: %d | Nome: %s\n", materiaPrima.getId(), materiaPrima.getNome());
45     System.out.printf("Quantidade disponível: %.2f %s\n", materiaPrima.getQuantidade(),
materiaPrima.getUnidade());
46     System.out.printf("Custo por unidade: R$ %.2f / %s\n", materiaPrima.getCustoPorUnidade(),
materiaPrima.getUnidade());
47     System.out.println("=====\n");
48 }
49
50 // Prints e Consultas
51 public void exibirBudget() {
52     System.out.printf("BUDGET ATUAL: R$ %.2f\n", budget);
53 }

```

5 Como deve ficar

tarefa-3 · final

```

1 public void exibirDemandas() {
2     Terminal.titulo("Pedidos registrados");
3     if (demandas.isEmpty()) {
4         Terminal.info("Nenhum pedido registrado.");
5         return;
6     }
7     System.out.printf(" %-4s %-20s %-13s %6s %-12s %12s %10s\n",
8         "#", "Medicamento", "Tipo", "Qtd", "Status", "Custo est.", "Insumo");
9     for (Demanda d : demandas) {
10         System.out.printf(" %-4d %-20s %-13s %6d %-12s %12s %8.1f kg\n",
11             d.getNumero(), d.getMedicamento(), d.getTipo().getNome(), d.getQuantidade(),
12             d.getStatus().getDescricao(), Terminal.moeda(d.calcularCustoEstimado()),
13             d.calcularConsumoEstimado());
14     }
15 }
16
17 /**
18  * Armazém de PRODUTOS ACABADOS, agrupado por lote. É diferente do estoque
19  * de matéria-prima (insumo bruto), que tem seu próprio método.
20  */
21 public void exibirArmazem() {
22     Terminal.titulo("Armazém de produtos acabados");
23     if (armazem.isEmpty()) {
24         Terminal.info("O armazém está vazio.");
25         return;
26     }
27     // LinkedHashMap mantém a ordem em que os lotes foram fabricados
28     Map<String, List<Produto>> porLote = new LinkedHashMap<>();
29     for (Produto produto : armazem) {
30         List<Produto> doLote = porLote.get(produto.getLote());
31         if (doLote == null) { // primeiro produto deste lote
32             doLote = new ArrayList<>();
33             porLote.put(produto.getLote(), doLote);
34         }
35         doLote.add(produto);
36     }
37     System.out.printf(" %-9s %-20s %-13s %5s %6s %6s %9s %7s %8s\n",
38         "Lote", "Medicamento", "Tipo", "Qtd", "Aprov", "Rejeit", "Qualidade", "Risco", "Em
risco");
39     for (Map.Entry<String, List<Produto>> entrada : porLote.entrySet()) {
40         List<Produto> lote = entrada.getValue();
41         Produto exemplo = lote.get(0);
42         int aprovados = 0;
43         int emRisco = 0;
44         double somaRisco = 0;

```

```

45         for (Produto p : lote) {
46             if (p.foiAprovado()) {
47                 aprovados++;
48             }
49             if (p.precisaManutencao()) {
50                 emRisco++;
51             }
52             somaRisco += p.getProbabilidadeFalhaAcumulada();
53         }
54         System.out.printf(" %-9s %-20s %-13s %5d %6d %6d %8.0f%% %6.1f%% %8d%n",
55             entrada.getKey(), exemplo.getNome(), exemplo.getTipo().getNome(), lote.size(),
56             aprovados, lote.size() - aprovados, exemplo.getQualidade() * 100,
57             somaRisco / lote.size() * 100, emRisco);
58     }
59     Terminal.info("Total no armazém: " + armazem.size() + " unidades em " + porLote.size() + "
lote(s).");
60 }
61
62 public void exibirEstoqueMateriaPrima() {
63     Terminal.titulo("Estoque de matéria-prima (insumos)");
64     Terminal.info(String.format("#%d %s", materiaPrima.getId(), materiaPrima.getNome()));
65     Terminal.info(String.format("Disponível: %.1f %s | Preço: %s/%s",
66         materiaPrima.getQuantidade(), materiaPrima.getUnidade(),
67         Terminal.moeda(materiaPrima.getCustoPorUnidade()), materiaPrima.getUnidade()));
68 }

```

Auditoria (usa a interface Auditavel) TAREFA 3

1 Lógica

Máquinas e produtos entram na **mesma** lista do tipo `Auditavel`. O laço chama os dois métodos da interface sem saber quem é quem: polimorfismo. Os detalhes de máquinas e produtos usam um método só, que aceita as duas listas.

2 Pseudocódigo

```

gerarAuditoriaGeral:
    itens <- máquinas + produtos (lista de Auditavel)
    para cada item: se precisaManutencao: conta e mostra o relatório
    mostra totais
detalharMaquinas / detalharProdutos: imprimirDiagnosticos(lista)
imprimirDiagnosticos(lista): para cada item: mostra o relatório

```

3 Código antigo (Tarefa 2)

Não existia na Tarefa 2.

4 Como está agora (seu arquivo)

Ainda não existe no seu `tarefa-3/src`.

5 Como deve ficar

tarefa-3 · final

```

1  /**
2   * Relatório consolidado. Máquinas e produtos entram na MESMA lista do tipo
3   * Auditavel; o laço não sabe (nem precisa saber) qual é qual.
4   */
5  public void gerarAuditoriaGeral() {
6      List<Auditavel> itens = new ArrayList<>();
7      itens.addAll(maquinas);
8      itens.addAll(armazem);
9
10     Terminal.titulo("Auditoria geral da planta");
11     int criticos = 0;
12     for (Auditavel item : itens) {
13         if (item.precisaManutencao()) {
14             criticos++;
15             Terminal.aviso(item.gerarRelatorio Diagnostico());

```

```

16     }
17     }
18     Terminal.info(String.format("Itens auditados: %d | exigem intervenção: %d | em conformidade: %d",
19         itens.size(), criticos, itens.size() - criticos));
20     if (criticos == 0) {
21         Terminal.sucesso("Planta em conformidade com as Boas Práticas de Fabricação.");
22     }
23 }
24
25 public void detalharMaquinas() {
26     Terminal.titulo("Diagnóstico das máquinas");
27     imprimirDiagnosticos(maquinas);
28 }
29
30 public void detalharProdutos() {
31     Terminal.titulo("Diagnóstico dos produtos");
32     imprimirDiagnosticos(armazem);
33 }
34
35 /** "? extends Auditavel" aceita List<Maquina> e List<Produto> (DRY). */
36 private void imprimirDiagnosticos(List<? extends Auditavel> itens) {
37     if (itens.isEmpty()) {
38         Terminal.info("Nada para exibir.");
39         return;
40     }
41     for (Auditavel item : itens) {
42         Terminal.info(item.gerarRelatorioDiagnostico());
43     }
44 }

```

Getters MUDA

1 Lógica

As listas saem como somente-leitura (`unmodifiableList`): quem está fora vê, mas não mexe. Os `set` saem.

2 Pseudocódigo

getters das listas (só leitura), estratégia, cenário, budget, matéria-prima e custo do reparo

3 Código antigo (Tarefa 2)

tarefa-2

```

1  // Getters e Setters
2  public ArrayList<Demanda> getDemandas() {
3      return demandas;
4  }
5
6  public void setDemandas(ArrayList<Demanda> demandas) {
7      this.demandas = demandas;
8  }
9
10 public ArrayList<Produto> getProdutosFabricados() {
11     return produtosFabricados;
12 }
13
14 public void setProdutosFabricados(ArrayList<Produto> produtosFabricados) {
15     this.produtosFabricados = produtosFabricados;
16 }
17
18 public ArrayList<Maquina> getMaquinas() {
19     return maquinas;
20 }
21
22 public void setMaquinas(ArrayList<Maquina> maquinas) {
23     this.maquinas = maquinas;
24 }
25
26 public MateriaPrima getMateriaPrima() {
27     return materiaPrima;

```

```

28     }
29
30     public void setMateriaPrima(MateriaPrima materiaPrima) {
31         this.materiaPrima = materiaPrima;
32     }
33
34     public double getBudget() {
35         return budget;
36     }
37
38     public void setBudget(double budget) {
39         this.budget = budget;
40     }
41 }

```

4 Como está agora (seu arquivo)

Igual ao código antigo (passo 3): ainda não foi mexido.

5 Como deve ficar

tarefa-3 · final

```

1     public List<Demanda> getDemandas() { return Collections.unmodifiableList(demandas); }
2     public List<Produto> getArmazem() { return Collections.unmodifiableList(armazem); }
3     public List<Maquina> getMaquinas() { return Collections.unmodifiableList(maquinas); }
4     public EstrategiaProducao getEstrategiaAtual() { return estrategiaAtual; }
5     public Cenario getCenario() { return cenario; }
6     public double getBudget() { return budget; }
7     public MateriaPrima getMateriaPrima() { return materiaPrima; }
8     public static double getCustoReparo() { return CUSTO_REPARO; }
9 }

```

Item 19 · ConfiguratorFabrica

X falta criar

ConfiguratorFabrica.java

Montar a fábrica a partir do cenário TAREFA 3

1 Lógica

Na Tarefa 2 o `Main` montava tudo com números fixos. Agora quem monta é esta classe, a partir do cenário escolhido: um `Random` com a semente do cenário, a matéria-prima com o estoque do cenário, as três máquinas e os pedidos iniciais. Separar isso deixa o menu só com o menu.

2 Pseudocódigo

```
montar(cenario, estrategia):
    random <- Random(semente do cenário)
    materiaPrima <- estoque inicial do cenário
    gerenciador <- novo(materiaPrima, cenario, estrategia)
    adiciona Processamento, Embalagem, Inspeção (com cenario e random)
    registra 4 pedidos iniciais
    devolve gerenciador
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 // 1. Instanciar Matéria-Prima e Gerenciador de Produção
2 MateriaPrima materiaPrima = new MateriaPrima(1, "Princípio Ativo Farmacêutico (Insumo)", 500.0,
"kg", 10.0);
3 GerenciadorProducao gerenciador = new GerenciadorProducao(materiaPrima, 1000.0);
4
5 // 2. Instanciar e registrar as Máquinas da linha de produção
6 MaquinaProcessamento maquinaProcessamento = new MaquinaProcessamento("Reator Farmacêutico /
Mistura", 500, 0.0, 1.50, 0.15);
7 MaquinaEmbalagem maquinaEmbalagem = new MaquinaEmbalagem("Embaladora Blister / Selagem", 500, 0.0,
1.00, 0.10);
8 MaquinaInspecao maquinaInspecao = new MaquinaInspecao("Espectrômetro de Inspeção / Controle
Anvisa", 500, 0.05, 2.00);
9
10 gerenciador.adicionarMaquina(maquinaProcessamento);
11 gerenciador.adicionarMaquina(maquinaEmbalagem);
12 gerenciador.adicionarMaquina(maquinaInspecao);
13
14 // 3. Registrar Demandas Iniciais de Medicamentos
15 gerenciador.registrarDemanda(new Demanda("Medicamento de Alta Qualidade", 50));
16 gerenciador.registrarDemanda(new Demanda("Medicamento de Média Qualidade", 50));
17 gerenciador.registrarDemanda(new Demanda("Medicamento de Baixa Qualidade", 50));
```

Era o começo do `main`.

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1 // 1. Instanciar Matéria-Prima e Gerenciador de Produção
2 MateriaPrima materiaPrima = new MateriaPrima(1, "Princípio Ativo Farmacêutico (Insumo)", 500.0,
"kg", 10.0);
3 GerenciadorProducao gerenciador = new GerenciadorProducao(materiaPrima, 1000.0);
4
5 // 2. Instanciar e registrar as Máquinas da linha de produção
6 MaquinaProcessamento maquinaProcessamento = new MaquinaProcessamento("Reator Farmacêutico /
Mistura", 500, 0.0, 1.50, 0.15);
7 MaquinaEmbalagem maquinaEmbalagem = new MaquinaEmbalagem("Embaladora Blister / Selagem", 500, 0.0,
1.00, 0.10);
8 MaquinaInspecao maquinaInspecao = new MaquinaInspecao("Espectrômetro de Inspeção / Controle
Anvisa", 500, 0.05, 2.00);
9
10 gerenciador.adicionarMaquina(maquinaProcessamento);
```

```

11 gerenciador.adicionarMaquina(maquinaEmbalagem);
12 gerenciador.adicionarMaquina(maquinaInspecao);
13
14 // 3. Registrar Demandas Iniciais de Medicamentos
15 gerenciador.registrarDemanda(new Demanda("Medicamento Controlado", 50));
16 gerenciador.registrarDemanda(new Demanda("Medicamento Contínuo", 50));
17 gerenciador.registrarDemanda(new Demanda("Medicamento Genérico", 50));

```

5 Como deve ficar

tarefa-3 · final

```

1  import java.util.Random;
2
3  /**
4   * Monta a fábrica de acordo com o cenário escolhido: matéria-prima, linha de
5   * máquinas e pedidos iniciais. Separar isso do menu e do gerenciador segue o
6   * SRP: cada classe tem um único motivo para mudar.
7   */
8  public final class ConfiguradorFabrica {
9
10     private static final double PRECO_PRINCIPIO_ATIVO = 10.0; // R$ por kg
11
12     private ConfiguradorFabrica() {
13     }
14
15     public static GerenciadorProducao montar(Cenario cenario, EstrategiaProducao estrategiaInicial) {
16         // Um único Random com a seed do cenário: mesma escolha = mesma simulação.
17         Random random = new Random(cenario.getSementeAleatoria());
18
19         MateriaPrima principioAtivo = new MateriaPrima(1, "Princípio Ativo Farmacêutico (IFA)",
20             cenario.getEstoqueInicialMateriaPrima(), "kg", PRECO_PRINCIPIO_ATIVO);
21
22         GerenciadorProducao gerenciador = new GerenciadorProducao(principioAtivo, cenario,
23             estrategiaInicial);
24
25         // Linha: Processamento -> Embalagem -> Inspeção (valores da Tarefa 2)
26         gerenciador.adicionarMaquina(new MaquinaProcessamento("Reator de Mistura", 500, 0.15, 1.50,
27             cenario, random));
28         gerenciador.adicionarMaquina(new MaquinaEmbalagem("Blistadeira", 500, 0.10, 1.00, cenario,
29             random));
30         gerenciador.adicionarMaquina(new MaquinaInspecao("Espectrômetro Anvisa", 500, 0.05, 2.00, cenario,
31             random));
32
33         // Pedidos iniciais: a ordem abaixo é a ordem de chegada (FIFO).
34         // Quantidades diferentes fazem cada estratégia escolher um pedido diferente.
35         gerenciador.registrarDemanda(TipoMedicamento.GENERICO, "Dipirona 500mg", 40);
36         gerenciador.registrarDemanda(TipoMedicamento.CONTROLADO, "Ritalina LA 40mg", 15);
37         gerenciador.registrarDemanda(TipoMedicamento.CONTINUO, "Losartana 50mg", 80);
38         gerenciador.registrarDemanda(TipoMedicamento.GENERICO, "Paracetamol 500mg", 120);
39
40         return gerenciador;
41     }
42 }

```

Item 20 · MenuFabrica

X falta criar

MenuFabrica.java

O menu único de 10 opções do Main vira um menu com **submenus** (o enunciado pede). Toda regra fica no gerenciador; o menu só lê a opção e chama o método certo.

Construtor e laço principal TAREFA 3

1 Lógica

Guarda as três estratégias numa lista. Ao iniciar: mostra a abertura, pede o cenário, monta a fábrica pelo configurador e entra no laço do menu principal.

2 Pseudocódigo

```
construtor: estrategias <- [Fila, Surto, Farmácia Popular]
iniciar:
  abertura ; cenario <- escolherCenario ; se nenhum: sai
  gerenciador <- ConfiguradorFabrica.montar(cenario, primeira estratégia)
  repete: cabeçalho ; opções 1 a 6 e 0 ; chama o submenu escolhido
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1      boolean executando = true;
2
3      while (executando) {
4          // Exibição do Menu
5          System.out.println("\n=====");
6          System.out.println("          FÁBRICA DE MEDICAMENTOS - Crescendo para te ajudar!!!");
7          System.out.println("          A vida é uma caixinha de surpresas, a nossa tem remédio barato
hehe. ");
8          System.out.println("          Desenvolvido por: Adriann e Alex");
9          System.out.println("=====");
10         System.out.printf("BUDGET ATUAL: R$ %.2f\n\n", gerenciador.getBudget());
11
12         System.out.println("ATUALIZAR DEMANDAS");
13         System.out.println("1 - Atualizar demanda de Medicamento de Alta Qualidade");
14         System.out.println("2 - Atualizar demanda de Medicamento de Média Qualidade");
15         System.out.println("3 - Atualizar demanda de Medicamento de Baixa Qualidade");
16
17         System.out.println("\nFABRICAR");
18         System.out.println("4 - Fabricar Medicamento de Alta Qualidade");
19         System.out.println("5 - Fabricar Medicamento de Média Qualidade");
20         System.out.println("6 - Fabricar Medicamento de Baixa Qualidade");
21
22         System.out.println("\nCONSULTAR");
23         System.out.println("7 - Ver armazém");
24         System.out.println("8 - Ver estoque de matéria-prima");
25
26         System.out.println("\nCOMPRAR MATÉRIA-PRIMA");
27         System.out.println("9 - Comprar matéria-prima");
28
29         System.out.println("\n0 - SAIR");
30         System.out.print("ESCOLHA: ");
```

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1      boolean executando = true;
2
3      while (executando) {
4          // Exibição do Menu
5          System.out.println("\n=====");
6          System.out.println("          FÁBRICA DE MEDICAMENTOS - Crescendo para te ajudar!!!");
```

```

7      System.out.println("          A vida é uma caixinha de surpresas, a nossa tem remédio barato
hehe. ");
8      System.out.println("          Desenvolvido por: Adriann e Alex");
9      System.out.println("=====");
10     System.out.printf("BUDGET ATUAL: R$ %.2f\n\n", gerenciador.getBudget());
11
12     System.out.println("ATUALIZAR DEMANDAS");
13     System.out.println("1 - Atualizar demanda de Medicamento Controlado");
14     System.out.println("2 - Atualizar demanda de Medicamento Contínuo");
15     System.out.println("3 - Atualizar demanda de Medicamento Genérico");
16
17     System.out.println("\nFABRICAR");
18     System.out.println("4 - Fabricar Medicamento Controlado");
19     System.out.println("5 - Fabricar Medicamento Contínuo");
20     System.out.println("6 - Fabricar Medicamento Genérico");
21
22     System.out.println("\nCONSULTAR");
23     System.out.println("7 - Ver demandas");
24     System.out.println("8 - Ver armazém");
25     System.out.println("9 - Ver estoque de matéria-prima");
26
27     System.out.println("\nCOMPRAR MATÉRIA-PRIMA");
28     System.out.println("10 - Comprar matéria-prima");
29
30     System.out.println("\n0 - SAIR");
31     System.out.print("ESCOLHA: ");

```

5 Como deve ficar

tarefa-3 · final

```

1  public class MenuFabrica {
2
3      private final Scanner scanner;
4      private final List<EstrategiaProducao> estrategias;
5      private GerenciadorProducao gerenciador;
6
7      public MenuFabrica(Scanner scanner) {
8          this.scanner = scanner;
9          // Todas as estratégias disponíveis; o gerenciador só recebe a interface.
10         this.estrategias = Arrays.asList(
11             new EstrategiaFilaDeReceitas(),
12             new EstrategiaSurtoEpidemico(),
13             new EstrategiaFarmaciaPopular());
14     }
15
16     public void iniciar() {
17         exibirAbertura();
18         Cenario cenario = escolherCenario();
19         if (cenario == null) {
20             Terminal.info("Nenhum cenário escolhido. Até logo!");
21             return;
22         }
23         gerenciador = ConfiguradorFabrica.montar(cenario, estrategias.get(0));
24         Terminal.sucesso("Cenário \"" + cenario.getNome() + "\" carregado. Bom turno!");
25
26         boolean executando = true;
27         while (executando) {
28             exibirCabecalho();
29             Terminal.opcao(1, "Pedidos (demandas)");
30             Terminal.opcao(2, "Fabricação");
31             Terminal.opcao(3, "Consultar armazém / matéria-prima");
32             Terminal.opcao(4, "Comprar matéria-prima");
33             Terminal.opcao(5, "Estratégia de produção");
34             Terminal.opcao(6, "Auditoria e manutenção");
35             Terminal.opcao(0, "Sair");
36             int escolha = Terminal.lerInteiro(scanner, "\n ESCOLHA: ", 0, 6);
37             switch (escolha) {
38                 case 1: menuPedidos(); break;
39                 case 2: menuFabricacao(); break;
40                 case 3: menuConsultas(); break;
41                 case 4: comprarMateriaPrima(); break;
42                 case 5: menuEstrategia(); break;

```

```

43         case 6: menuAuditoria(); break;
44         default:
45             executando = false;
46             Terminal.info("Encerrando o turno da fábrica. Até logo!");
47     }
48 }
49 }

```

Telas: abertura, cenário, cabeçalho TAREFA 3

1 Lógica

O cabeçalho mostra sempre estratégia ativa, cenário, caixa e máquinas funcionando, como pede o enunciado. A escolha de cenário lista os valores de cada um.

2 Pseudocódigo

```

exibirAbertura: título e créditos
escolherCenario: lista os cenários com seus números ; lê a opção ; devolve o cenário ou null
exibirCabeçalho: estratégia, cenário, budget, máquinas operantes

```

3 Código antigo (Tarefa 2)

Só o título e o budget, impressos no início de cada volta do laço (bloco anterior).

4 Como está agora (seu arquivo)

Ainda não existe no seu `tarefa-3/src`.

5 Como deve ficar

tarefa-3 · final

```

1  private void exibirAbertura() {
2      Terminal.titulo("Fábrica de Medicamentos - crescendo para te ajudar!");
3      Terminal.info("A vida é uma caixinha de surpresas; a nossa tem remédio barato.");
4      Terminal.info("Desenvolvido por: Adriann e Alex");
5  }
6
7  private Cenario escolherCenario() {
8      Terminal.secao("Escolha o cenário de operação");
9      Cenario[] cenarios = Cenario.values();
10     for (int i = 0; i < cenarios.length; i++) {
11         Cenario c = cenarios[i];
12         Terminal.opcao(i + 1, c.getNome() + " - " + c.getDescricao());
13         Terminal.info(String.format("          caixa %s | insumo %.0f kg | falhas x%.1f | desgaste %.0f-
%.0f por lote",
14                                     Terminal.moeda(c.getBudgetInicial()), c.getEstoqueInicialMateriaPrima(),
15                                     c.getFatorFalha(), c.getDesgasteMinimo(), c.getDesgasteMaximo()));
16     }
17     Terminal.opcao(0, "Sair");
18     int escolha = Terminal.lerInteiro(scanner, "\n CENÁRIO: ", 0, cenarios.length);
19     return escolha == 0 ? null : cenarios[escolha - 1];
20 }
21
22 private void exibirCabeçalho() {
23     System.out.println();
24     System.out.println(Terminal.CIANO + Terminal.linha('=') + Terminal.RESET);
25     System.out.println(Terminal.NEGRITO + " [FÁBRICA DE MEDICAMENTOS]" + Terminal.RESET);
26     System.out.println("  ESTRATÉGIA ATUAL: " + Terminal.AMARELO
27         + gerenciador.getEstrategiaAtual().getNomeEstrategia() + Terminal.RESET);
28     System.out.println("  CENÁRIO ATIVO:    " + gerenciador.getCenario().getNome());
29     System.out.println("  BUDGET ATUAL:     " + Terminal.VERDE
30         + Terminal.moeda(gerenciador.getBudget()) + Terminal.RESET);
31     System.out.printf("  LINHA:           %d/%d máquinas operantes%n",
32         gerenciador.contarMaquinasOperantes(), gerenciador.getMaquinas().size());
33     System.out.println(Terminal.CIANO + Terminal.linha('=') + Terminal.RESET);
34 }

```

Submenu de pedidos TAREFA 3

1 Lógica

As opções 1, 2 e 3 do Main (uma por tipo, cada uma repetindo a leitura) viram um submenu: listar, registrar novo, atualizar e cancelar, escolhendo o pedido pelo número.

2 Pseudocódigo

```
menuPedidos: repete: 1 listar | 2 registrar | 3 atualizar | 4 cancelar | 0 volta
registrarPedido: escolhe o tipo ; lê nome e quantidade ; gerenciador.registrarDemanda
atualizarPedido: lista ; lê número e quantidade ; gerenciador.atualizarDemanda
cancelarPedido: lista ; lê número ; gerenciador.cancelarDemanda
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1         case 1:
2             System.out.print("Digite a nova quantidade para a demanda de Alta Qualidade: ");
3             if (scanner.hasNextInt()) {
4                 int qtd = scanner.nextInt();
5                 gerenciador.atualizarDemanda(0, qtd);
6             } else {
7                 System.out.println("Entrada inválida! Digite apenas números inteiros.");
8                 scanner.next();
9             }
10            break;
```

(case 2 e case 3 eram iguais, só mudando o índice)

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1         case 1:
2             System.out.print("Digite a nova quantidade para a demanda de Medicamento Controlado: ");
3             if (scanner.hasNextInt()) {
4                 int qtd = scanner.nextInt();
5                 gerenciador.atualizarDemanda(0, qtd);
6             } else {
7                 System.out.println("Entrada inválida! Digite apenas números inteiros.");
8                 scanner.next();
9             }
10            break;
```

5 Como deve ficar

tarefa-3 · final

```
1     private void menuPedidos() {
2         boolean noSubMenu = true;
3         while (noSubMenu) {
4             Terminal.secao("Pedidos");
5             Terminal.opcao(1, "Listar pedidos (status via enum)");
6             Terminal.opcao(2, "Registrar novo pedido");
7             Terminal.opcao(3, "Atualizar quantidade de um pedido");
8             Terminal.opcao(4, "Cancelar pedido");
9             Terminal.opcao(0, "Voltar");
10            switch (Terminal.lerInteiro(scanner, " ESCOLHA: ", 0, 4)) {
11                case 1: gerenciador.exibirDemandas(); break;
12                case 2: registrarPedido(); break;
13                case 3: atualizarPedido(); break;
14                case 4: cancelarPedido(); break;
15                default: noSubMenu = false;
16            }
17        }
18    }
19 }
```

```

20     private void registrarPedido() {
21         TipoMedicamento[] tipos = TipoMedicamento.values();
22         for (int i = 0; i < tipos.length; i++) {
23             Terminal.opcao(i + 1, tipos[i].getNome() + String.format(" (%.0f kg de insumo por unidade)",
24                 tipos[i].getConsumoPorUnidade()));
25         }
26         int t = Terminal.lerInteiro(scanner, " Tipo (0 cancela): ", 0, tipos.length);
27         if (t == 0) {
28             return;
29         }
30         String nome = Terminal.lerTexto(scanner, " Nome do medicamento (ex.: Dipirona 1g): ");
31         if (nome.isEmpty()) {
32             nome = tipos[t - 1].getNome();
33         }
34         int quantidade = Terminal.lerInteiro(scanner, " Quantidade (0 cancela): ", 0, 100000);
35         if (quantidade == 0) {
36             return;
37         }
38         Demanda nova = gerenciador.registrarDemanda(tipos[t - 1], nome, quantidade);
39         Terminal.sucesso("Registrado: " + nova);
40     }
41
42     private void atualizarPedido() {
43         gerenciador.exibirDemandas();
44         int numero = Terminal.lerInteiro(scanner, " Número do pedido (0 cancela): ", 0,
Integer.MAX_VALUE);
45         if (numero == 0) {
46             return;
47         }
48         int quantidade = Terminal.lerInteiro(scanner, " Nova quantidade: ", 0, 100000);
49         gerenciador.atualizarDemanda(numero, quantidade);
50     }
51
52     private void cancelarPedido() {
53         gerenciador.exibirDemandas();
54         int numero = Terminal.lerInteiro(scanner, " Número do pedido a cancelar (0 volta): ", 0,
Integer.MAX_VALUE);
55         if (numero != 0) {
56             gerenciador.cancelarDemanda(numero);
57         }
58     }

```

Submenu de fabricação TAREFA 3

1 Lógica

As opções 4, 5 e 6 viram: processar o próximo pedido **pela estratégia**, ou fabricar um pedido específico pelo número.

2 Pseudocódigo

```

menuFabricacao: repete:
    1 -> gerenciador.executarProximaProducao
    2 -> lista ; lê número ; gerenciador.fabricarDemanda(numero)
    0 -> volta

```

3 Código antigo (Tarefa 2)

tarefa-2

```

1         case 4:
2             System.out.println("\nIniciando fabricação de Medicamento de Alta Qualidade...");
3             gerenciador.fabricarDemanda(0);
4             break;

```

(case 5 e case 6 iguais, com índices 1 e 2)

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```

1         case 4:
2             System.out.println("\nIniciando fabricação de Medicamento Controlado...");
3             gerenciador.fabricarDemanda(0);
4             break;

```

5 Como deve ficar

tarefa-3 · final

```

1     private void menuFabricacao() {
2         boolean noSubMenu = true;
3         while (noSubMenu) {
4             Terminal.secao("Fabricação");
5             Terminal.opcao(1, "Processar próximo pedido (usa a estratégia ativa)");
6             Terminal.opcao(2, "Fabricar um pedido específico");
7             Terminal.opcao(0, "Voltar");
8             switch (Terminal.lerInteiro(scanner, " ESCOLHA: ", 0, 2)) {
9                 case 1:
10                    gerenciador.executarProximaProducao();
11                    break;
12                 case 2:
13                    gerenciador.exibirDemandas();
14                    int numero = Terminal.lerInteiro(scanner, " Número do pedido (0 volta): ", 0,
Integer.MAX_VALUE);
15                    if (numero != 0) {
16                        gerenciador.fabricarDemanda(numero);
17                    }
18                    break;
19                 default:
20                    noSubMenu = false;
21             }
22         }
23     }

```

Consultas e compra de matéria-prima MUDA

1 Lógica

Armazém e estoque ficam num submenu de consultas; a compra usa o `Terminal.lerDecimal` em vez da validação manual.

2 Pseudocódigo

```

menuConsultas: 1 armazém | 2 estoque | 0 volta
comprarMateriaPrima: mostra estoque ; lê kg ; se > 0: gerenciador.comprarMateriaPrima

```

3 Código antigo (Tarefa 2)

tarefa-2

```

1         case 7:
2             gerenciador.exibirArmazem();
3             break;
4         case 8:
5             gerenciador.exibirEstoqueMateriaPrima();
6             break;
7         case 9:
8             System.out.print("Digite a quantidade de matéria-prima a ser comprada (kg): ");
9             if (!scanner.hasNextDouble()) {
10                System.out.println("Entrada inválida! Digite apenas números.");
11                scanner.next();
12            } else {
13                double quantidade = scanner.nextDouble();
14                gerenciador.comprarMateriaPrima(quantidade);
15            }
16            break;

```

4 Como está agora (seu arquivo)

tarefa-3 · hoje

```
1         case 7:
2             gerenciador.exibirDemandas();
3             break;
4         case 8:
5             gerenciador.exibirArmazem();
6             break;
7         case 9:
8             gerenciador.exibirEstoqueMateriaPrima();
9             break;
10        case 10:
11            System.out.print("Digite a quantidade de matéria-prima a ser comprada (kg): ");
12            if (!scanner.hasNextDouble()) {
13                System.out.println("Entrada inválida! Digite apenas números.");
14                scanner.next();
15            } else {
16                double quantidade = scanner.nextDouble();
17                gerenciador.comprarMateriaPrima(quantidade);
18            }
19            break;
```

5 Como deve ficar

tarefa-3 · final

```
1     private void menuConsultas() {
2         boolean noSubmenu = true;
3         while (noSubmenu) {
4             Terminal.secao("Consultar");
5             Terminal.opcao(1, "Ver armazém (produtos acabados)");
6             Terminal.opcao(2, "Ver estoque de matéria-prima");
7             Terminal.opcao(0, "Voltar");
8             switch (Terminal.lerInteiro(scanner, " ESCOLHA: ", 0, 2)) {
9                 case 1: gerenciador.exibirArmazem(); break;
10                case 2: gerenciador.exibirEstoqueMateriaPrima(); break;
11                default: noSubmenu = false;
12            }
13        }
14    }
15
16    private void comprarMateriaPrima() {
17        gerenciador.exibirEstoqueMateriaPrima();
18        double kg = Terminal.lerDecimal(scanner, " Quantos kg comprar? (0 cancela): ");
19        if (kg > 0) {
20            gerenciador.comprarMateriaPrima(kg);
21        }
22    }
```

Submenu de estratégia TAREFA 3

1 Lógica

Lista as três estratégias, marca a ativa e troca pela escolhida com `setEstrategia`, com o programa rodando.

2 Pseudocódigo

```
menuEstrategia: para cada estratégia: mostra nome (+ "<- ativa") ; lê opção ; gerenciador.setEstrategia
```

3 Código antigo (Tarefa 2)

Não existia na Tarefa 2.

4 Como está agora (seu arquivo)

Ainda não existe no seu `tarefa-3/src`.

5 Como deve ficar

tarefa-3 · final

```
1 private void menuEstrategia() {
2     Terminal.secao("Estratégia de produção");
3     for (int i = 0; i < estrategias.size(); i++) {
4         EstrategiaProducao e = estrategias.get(i);
5         String marca = (e == gerenciador.getEstrategiaAtual()) ? " <- ativa" : "";
6         Terminal.opcao(i + 1, e.getNomeEstrategia() + marca);
7     }
8     Terminal.opcao(0, "Voltar");
9     int escolha = Terminal.lerInteiro(scanner, " ESCOLHA: ", 0, estrategias.size());
10    if (escolha != 0) {
11        gerenciador.setEstrategia(estrategias.get(escolha - 1));
12    }
13 }
```

Submenu de auditoria e manutenção TAREFA 3

1 Lógica

Relatório geral (a lista de Auditavel), detalhes de máquinas e produtos, e reparo de máquina.

2 Pseudocódigo

```
menuAuditoria: 1 geral | 2 máquinas | 3 produtos | 4 reparar | 0 volta
repararMaquina: lista máquinas com saúde ; lê número ; gerenciador.repararMaquina
```

3 Código antigo (Tarefa 2)

Não existia na Tarefa 2.

4 Como está agora (seu arquivo)

Ainda não existe no seu tarefa-3/src.

5 Como deve ficar

tarefa-3 · final

```
1 private void menuAuditoria() {
2     boolean noSubmenu = true;
3     while (noSubmenu) {
4         Terminal.secao("Auditoria e manutenção");
5         Terminal.opcao(1, "Relatório geral (interface Auditavel)");
6         Terminal.opcao(2, "Detalhar máquinas");
7         Terminal.opcao(3, "Detalhar produtos");
8         Terminal.opcao(4, "Reparar máquina (" + Terminal.moeda(GerenciadorProducao.getCustoReparo()) +
9     ");");
10    Terminal.opcao(0, "Voltar");
11    switch (Terminal.lerInteiro(scanner, " ESCOLHA: ", 0, 4)) {
12        case 1: gerenciador.gerarAuditoriaGeral(); break;
13        case 2: gerenciador.detalharMaquinas(); break;
14        case 3: gerenciador.detalharProdutos(); break;
15        case 4: repararMaquina(); break;
16        default: noSubmenu = false;
17    }
18 }
19
20 private void repararMaquina() {
21     List<Maquina> maquinas = gerenciador.getMaquinas();
22     for (int i = 0; i < maquinas.size(); i++) {
23         Maquina m = maquinas.get(i);
24         Terminal.opcao(i + 1, String.format("%s (saúde %.1f%%, %s)",
25             m.getNome(), m.getSaude(), m.getEstado().getDescricao()));
26     }
27     int escolha = Terminal.lerInteiro(scanner, " Máquina (0 volta): ", 0, maquinas.size());
28     if (escolha != 0) {
29         gerenciador.repararMaquina(escolha - 1);
30     }
31 }
```

```
30     }  
31 }
```

Item 21 · Main

X ainda da Tarefa 2

Main.java

O ponto de entrada MUDA

1 Lógica

Todo o conteúdo do Main antigo foi distribuído: montagem no ConfiguradorFabrica, menu no MenuFabrica, leitura no Terminal. Sobra só abrir o Scanner, iniciar o menu e fechar.

2 Pseudocódigo

```
main:
  scanner <- novo Scanner
  new MenuFabrica(scanner).iniciar()
  fecha o scanner
```

3 Código antigo (Tarefa 2)

tarefa-2

```
1 public class Main {
2     public static void main(String[] args) {
3         Scanner scanner = new Scanner(System.in);
4         // 1. matéria-prima, gerenciador, máquinas e demandas (números fixos)
5         // 2. while (executando) { menu com 10 opções em um switch }
6         scanner.close();
7     }
8 }
```

Resumo: o arquivo original tem cerca de 150 linhas.

4 Como está agora (seu arquivo)

Igual à Tarefa 2 (só com os nomes novos dos medicamentos e a opção "Ver demandas"). Todo o conteúdo sai.

5 Como deve ficar

tarefa-3 · final

```
1 import java.util.Scanner;
2
3 /**
4  * Ponto de entrada. Fica pequeno de propósito: quem cuida do menu é o
5  * MenuFabrica e quem cuida da fábrica é o GerenciadorProducao.
6  */
7 public class Main {
8
9     public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11         new MenuFabrica(scanner).iniciar();
12         scanner.close();
13     }
14 }
```

8. Compilar, testar e entregar

[↑ voltar ao sumário](#)

8.1 Durante as etapas

Até a etapa E terminar, o `GerenciadorProducao` e o `Main` antigos não compilam. Compilem só o que já está pronto, por exemplo depois da etapa C:

```
cd tarefa-3
javac -d bin src/Status*.java src/Auditavel.java src/Cenario.java src/TipoMedicamento.java \
    src/Produto.java src/Medicamento*.java src/Demanda.java src/Maquina*.java src/MateriaPrima.java
```

8.2 No fim

```
cd tarefa-3
javac -d bin src/*.java
java -cp bin Main
```

Roteiro mínimo no menu: escolher a Pandemia; em Pedidos, listar; em Fabricação, processar pelo Fila de Receitas (vai faltar insumo); comprar 200 kg; processar de novo; trocar a estratégia para Farmácia Popular e processar; ver o armazém; abrir a Auditoria e reparar a máquina com menor saúde.

8.3 Entrega

```
git add tarefa-3
git commit -m "Tarefa 3: Strategy, enums, Auditavel, cenários e desgaste"
git push
gh release create tarefa3-RA1-RA2 --title "Tarefa 3" --notes "Entrega da Tarefa 3"
```

Troquem RA1-RA2 pelos RAs de vocês, atualizem a `justificativa.txt` (estratégias, cenários, por que cada enum, uso de IA) e enviem o link no Google Forms.